Lecture 1 — Framing and decomposing work for a coding agent

↑ Retour à « Lectures »

Lecture 1 — Framing and decomposing work for a coding agent

1 September · 1h30 · full cohort · immediately followed by Lab 1

Serves: C1. **One idea to land:** *the agent is not the bottleneck; the ambiguity in your request is.*

This is the lecture, written down: one section per slide, the picture that was on screen, and what was said about it. Read it after the room if you were there, and instead of the room if you were not.

Sycophancy: an agent agreeing instead of aligning

Give an agent a realistically fuzzy request — *"add caching to the ingestion step"* — against a real repository, and it will do something plausible, confidently, and wrong: wrong layer, wrong invalidation, wrong assumption about what "ingestion" means.

> The fastest way to waste an agent run is to hand it a fuzzy idea and say "go build this." It will build something. That is the problem.

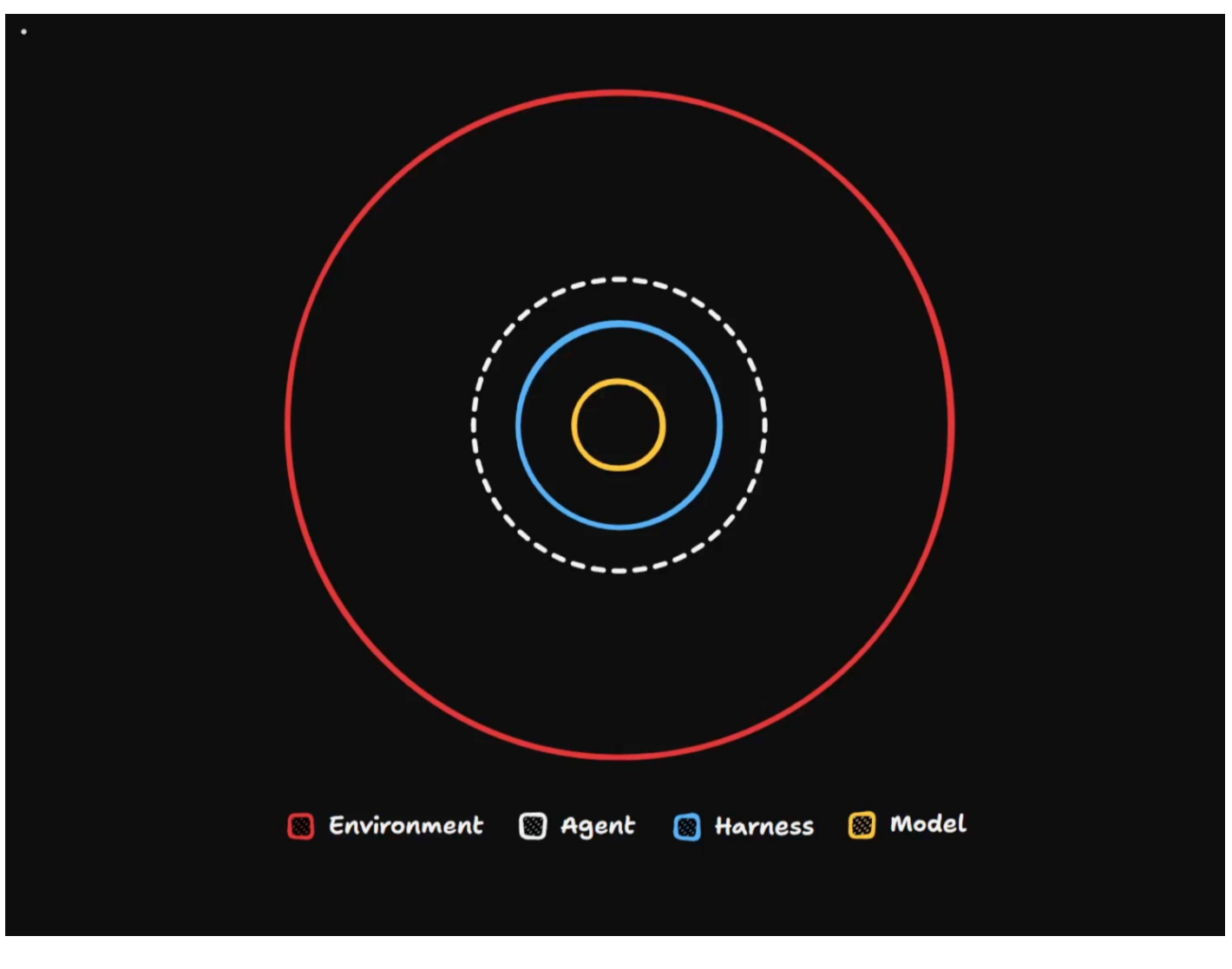
Whose fault is that? Nobody had decided what the request meant, so the agent decided for you. It always decides; the only question is whether you were in the room when it did. The name for this is **sycophancy** — you named a thing you wanted and it produced the thing, without ever stopping to check you agreed on what it should look like. It is not hallucination (nothing was invented), it is not non-determinism (it would behave the same way on any run), and it is not the context degrading (it went wrong immediately). The whole first half of this lecture is the countermeasure.

Parametric and contextual knowledge

Parametric
Contextual

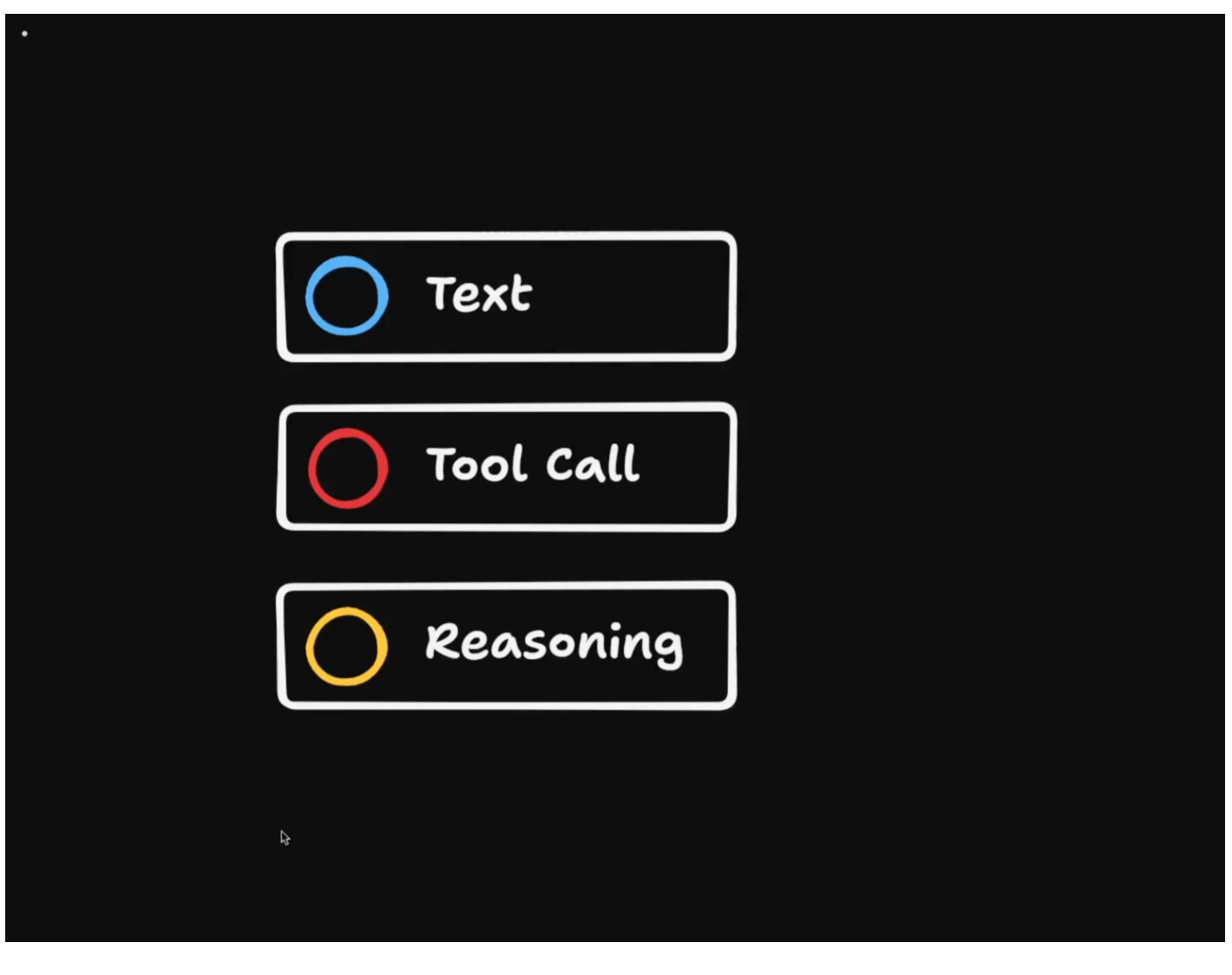
Parametric knowledge is baked into the weights: how Python works, what a left join is. **Contextual** knowledge is what you put in the window: your schema, your conventions, this ticket. The agent cannot tell you which is which, and it is equally confident about both — that is the mechanism behind most hallucinations you will hit. The practical rule: anything specific to your project, your company, or the last eighteen months has to be contextual, and if you did not put it there, it was guessed.

What an agent is made of



The **model** predicts tokens; it has no hands. The **harness** wraps it in a loop and a set of tools — Claude Code, Cursor and Codex are harnesses. Together they are the **agent**, and the agent runs inside an **environment**: a repository, a shell, a filesystem, credentials. When a run fails, ask which of those four failed. Almost nothing in this course is done to the model — you change the harness and you change the environment, which is why "which model should I use" is the least interesting question here and gets one section near the end.

The three things a model can emit



Text, a tool call, and reasoning — and no others. Everything an agent **does** is a tool call, so the harness's tool list is the exact boundary of what it can do. That is also the setup for Lecture 2: a hook intercepts a tool call, which is why enforcement works without the model's cooperation.

> This is the frame for the whole course: you are not learning to prompt. You are learning to remove the decisions the agent would otherwise make on your behalf.

Sharpening by interview: it asks, you decide

A fuzzy request becomes buildable by having the ambiguity **extracted from you**, question by question, before any code exists.

The move is to make the agent interview *you*. It asks; you decide. Facts are its job — it can read the repository, check what exists, look things up. Decisions are yours, and they are the only thing it cannot do. The shape is rounds of questions, each round's answers unlocking the next; in Claude Code it is a skill (`/grill-me`, `/grill-with-docs`), and in any harness it is a prompt that forbids the agent from writing code until it has asked. You know it is working when the questions are ones you could not have answered without thinking — that is the ambiguity becoming visible.

Low-fidelity and high-fidelity questions

Questions
Low Fidelity
High Fidelity

Low-fidelity questions settle the shape of the thing; high-fidelity ones settle its details. Asking a high-fidelity question before the shape exists produces an answer you will throw away — you cannot pick a column type for a table nobody has agreed to build.

Grillable and ungrillable questions

Grillable
Ungrillable

Some questions are **grillable**: there is a decision, you hold it, and being asked forces you to take it. Others are **ungrillable** — a fact the agent could have looked up, a thing genuinely nobody knows yet, or pure taste with no consequence. Grilling those burns context and produces false confidence. Knowing which is which is the Outstanding-band version of this skill.

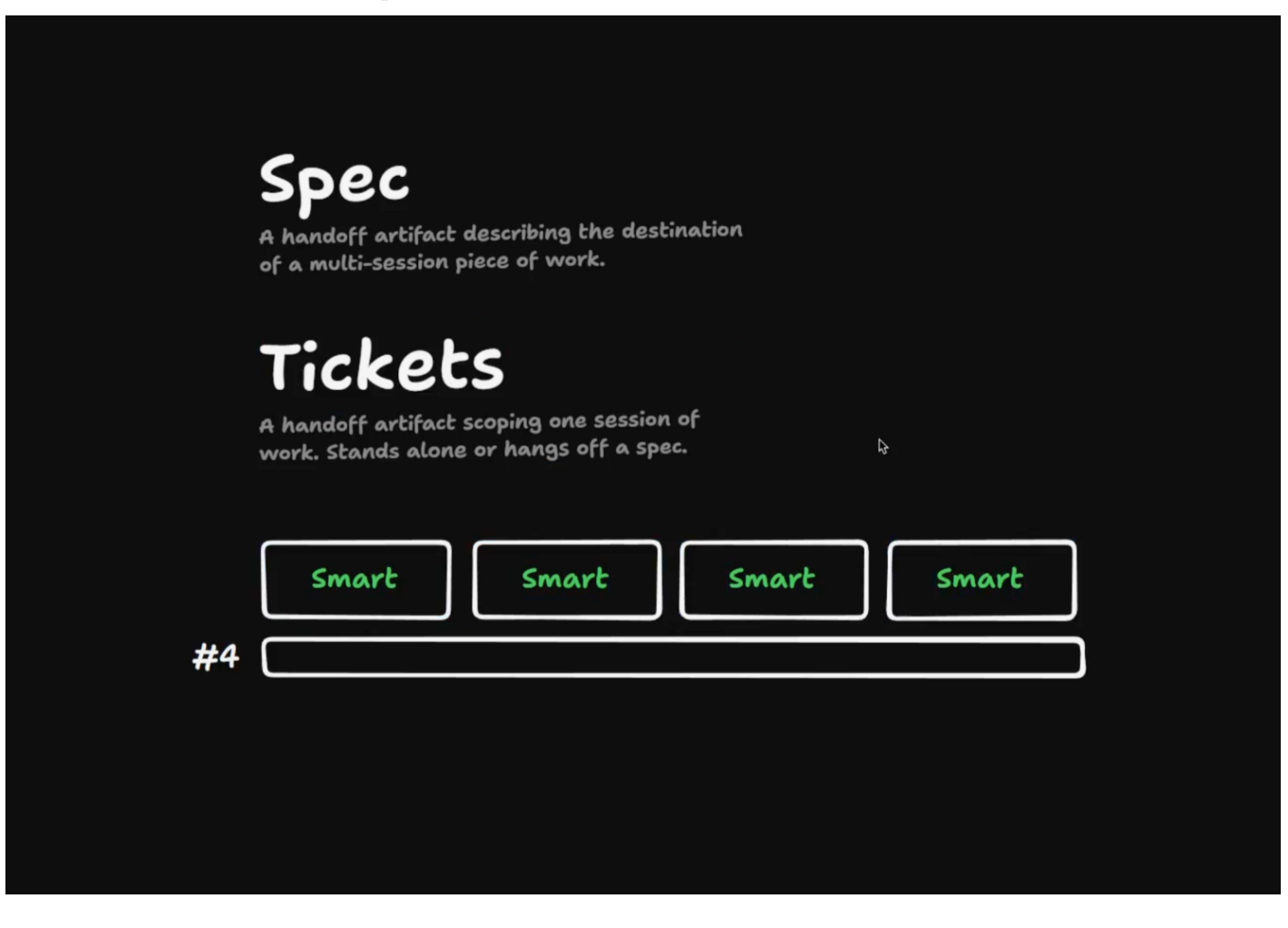
The grid rewards a specification that resolves an ambiguity. If your spec says the same thing as the ticket you started with, you have not done this step. (Grid: C1 Basic → Solid.)

The specification: the interview, written down

> Capture the shape...

A specification is what is being built, the decisions taken, and what was explicitly ruled out. That last part is what makes it a specification rather than a description. It lives wherever the project already keeps such things — an issue tracker, a markdown file — and not in the chat. Writing it down matters more with an agent than without one: the agent has no memory between sessions, so the document *is* its memory.

A spec and a ticket are two documents, and the difference is scope

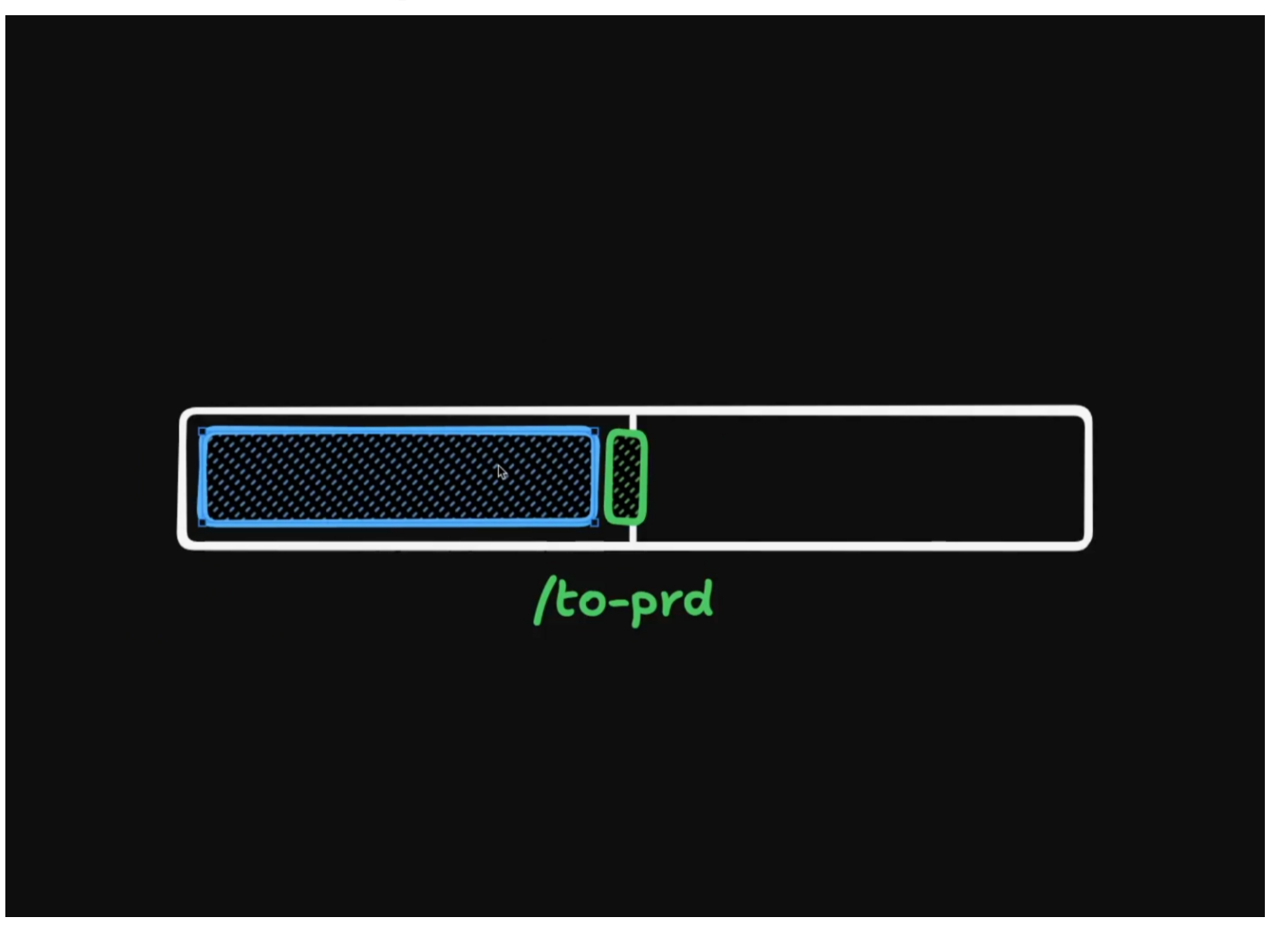


A handoff artifact describing the destination of a multi-session piece of work.

A handoff artifact scoping one session of work. Stands alone or hangs off a spec.

A **spec** describes the destination of the whole change, across however many sessions it takes. A **ticket** scopes exactly one session, and either stands alone or hangs off a spec. Both are handoff artifacts: their job is to survive the session that produced them.

Do not write the spec in the dumb zone

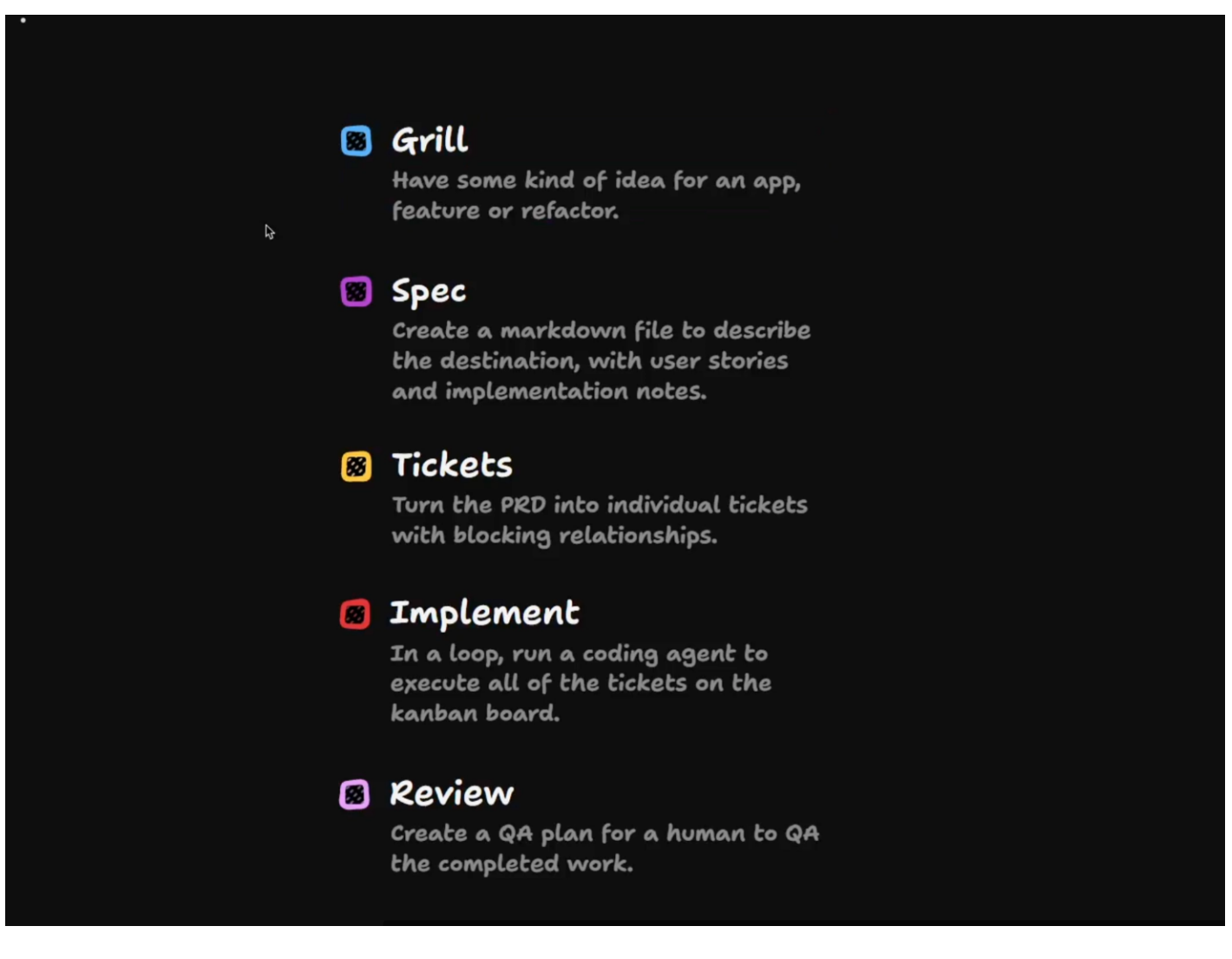


The interview is what fills a context window fastest — rounds of questions, repository reads, half-formed options. Writing the specification at the end of that conversation means writing the most important document of the whole change in the worst context you will have all day. Compress the interview into a document and start the spec in a fresh session. "Dumb zone" is made precise later in this lecture.

Close the spec, do not enshrine it

When the last ticket merges, archive the spec where it was written — a closed issue, a `docs/` entry marked superseded. It survives as a record of what was decided, without sitting in the repository presenting itself as documentation of how the system works. An agent that finds a committed spec stops there and never reads the code, which is the one document that cannot lie about itself. This is the same instinct as the primary-versus-secondary-source argument further down.

The whole method, if you want the map



Grill
Have some kind of idea for an app, feature or refactor.

Spec
Create a markdown file to describe the destination, with user stories and implementation notes.

Tickets
Turn the PRD into individual tickets with blocking relationships.

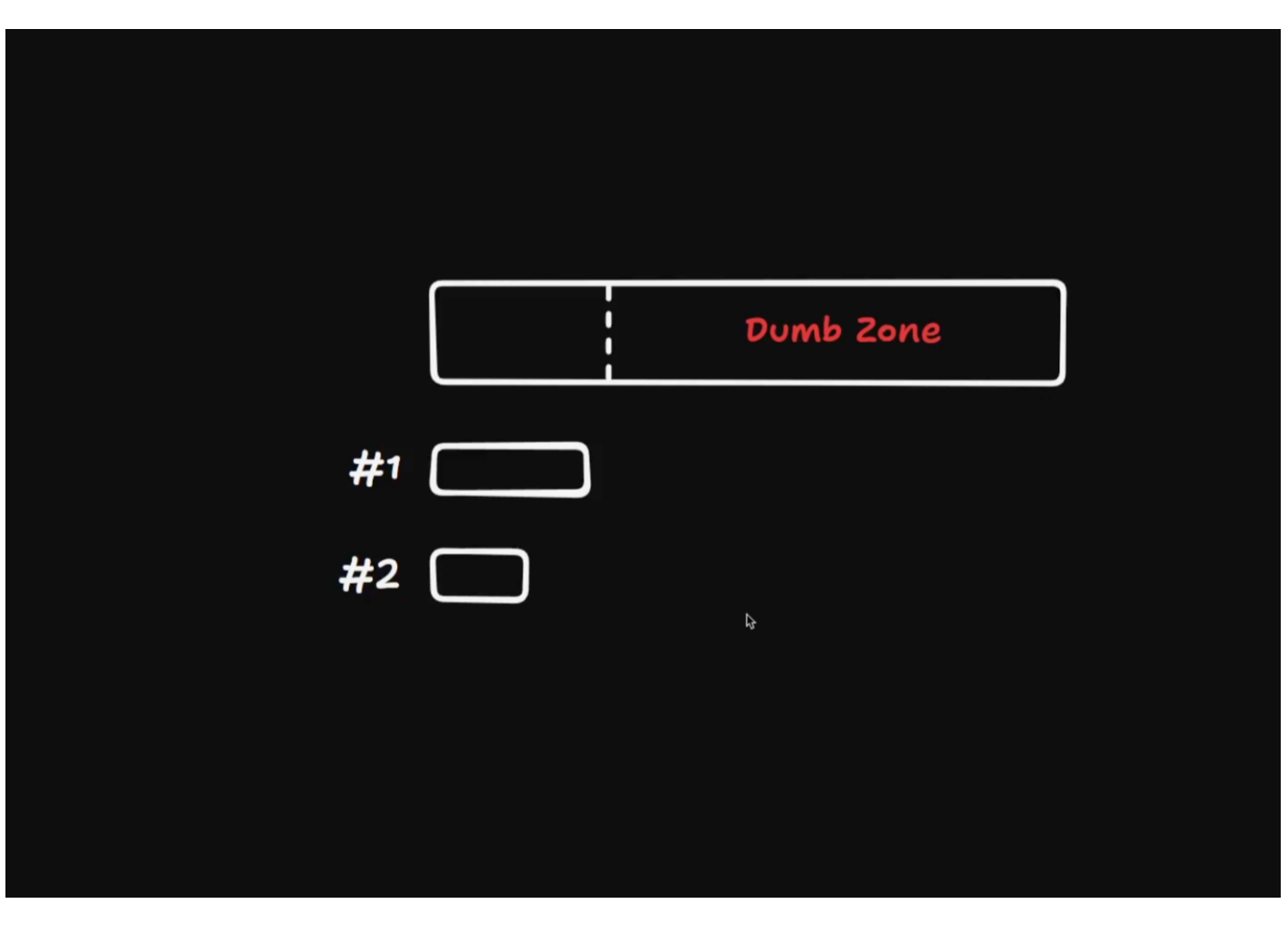
Implement
In a loop, run a coding agent to execute all of the tickets on the kanban board.

Review
Create a QA plan for a human to QA the completed work.

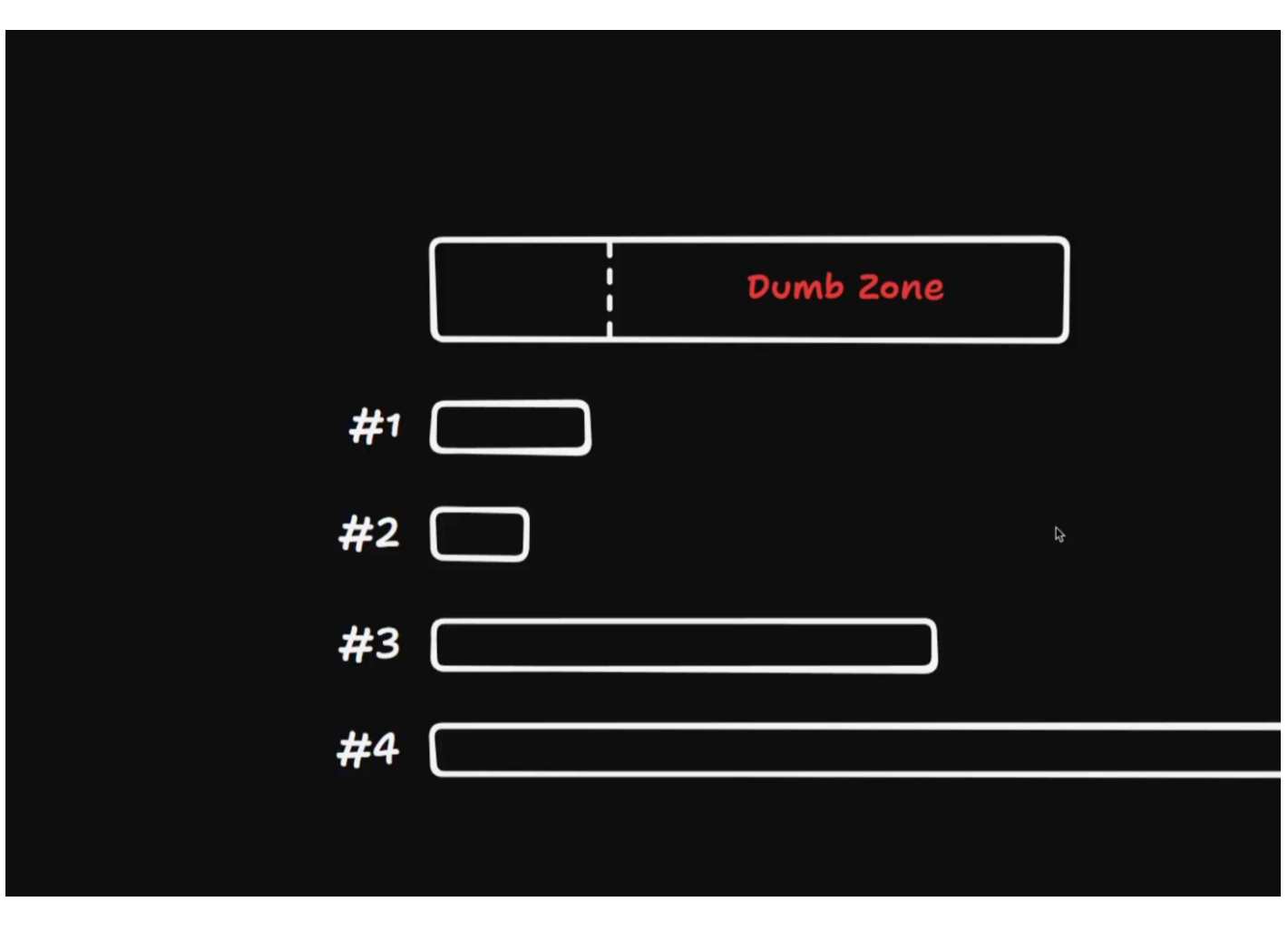
Grill the idea, write the destination down, cut it into tickets, execute them, review. Lab 1 is the first three; the 3 September autonomy slot is the fourth. The phases are the point, not the tooling around them.

Why the work has to be cut at all

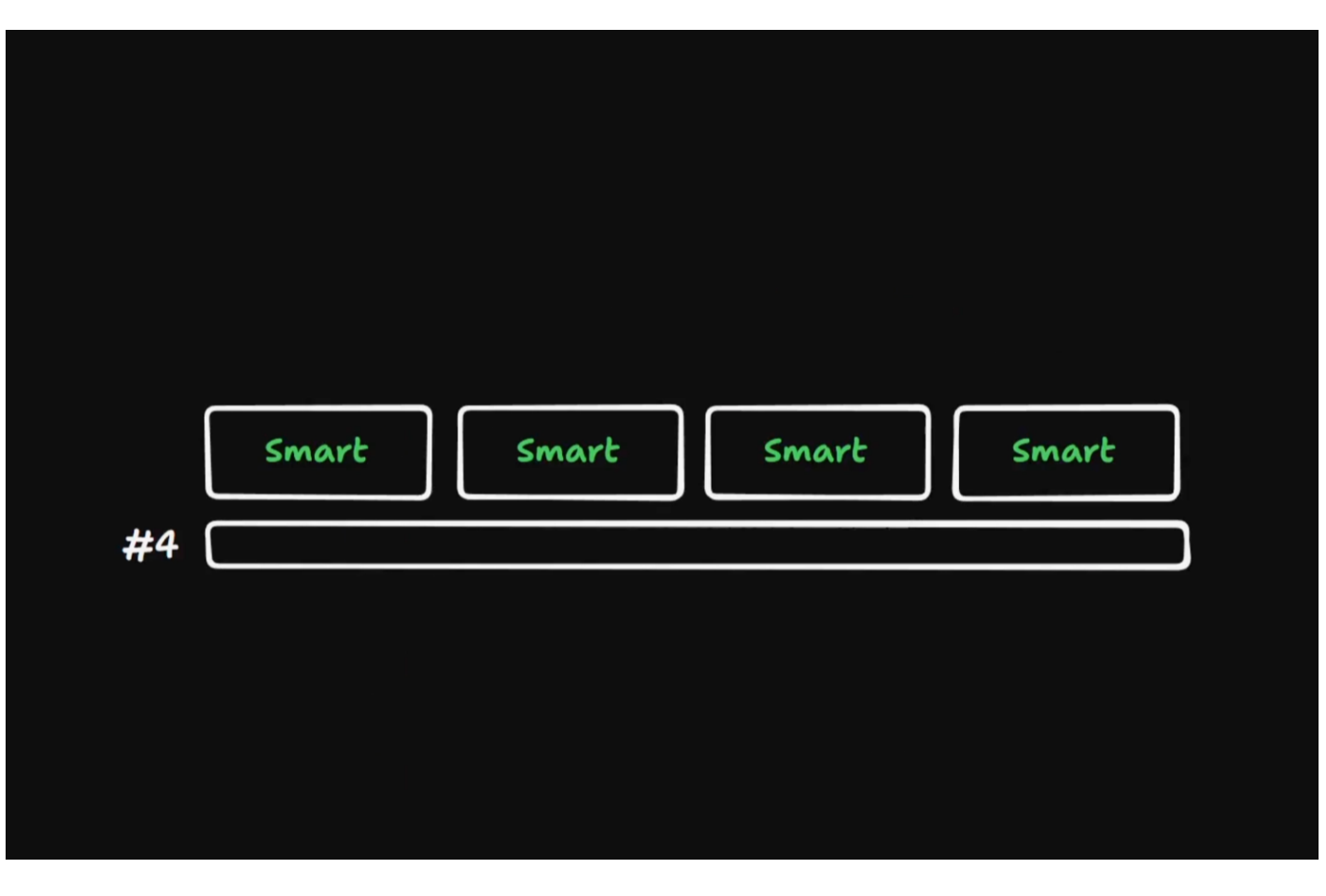
Decomposition is not project-management hygiene. It is a context constraint.



A small feature or a bug fix fits in one session, comfortably inside the smart zone. This is why agents feel excellent on small work — and why the experience does not generalise.

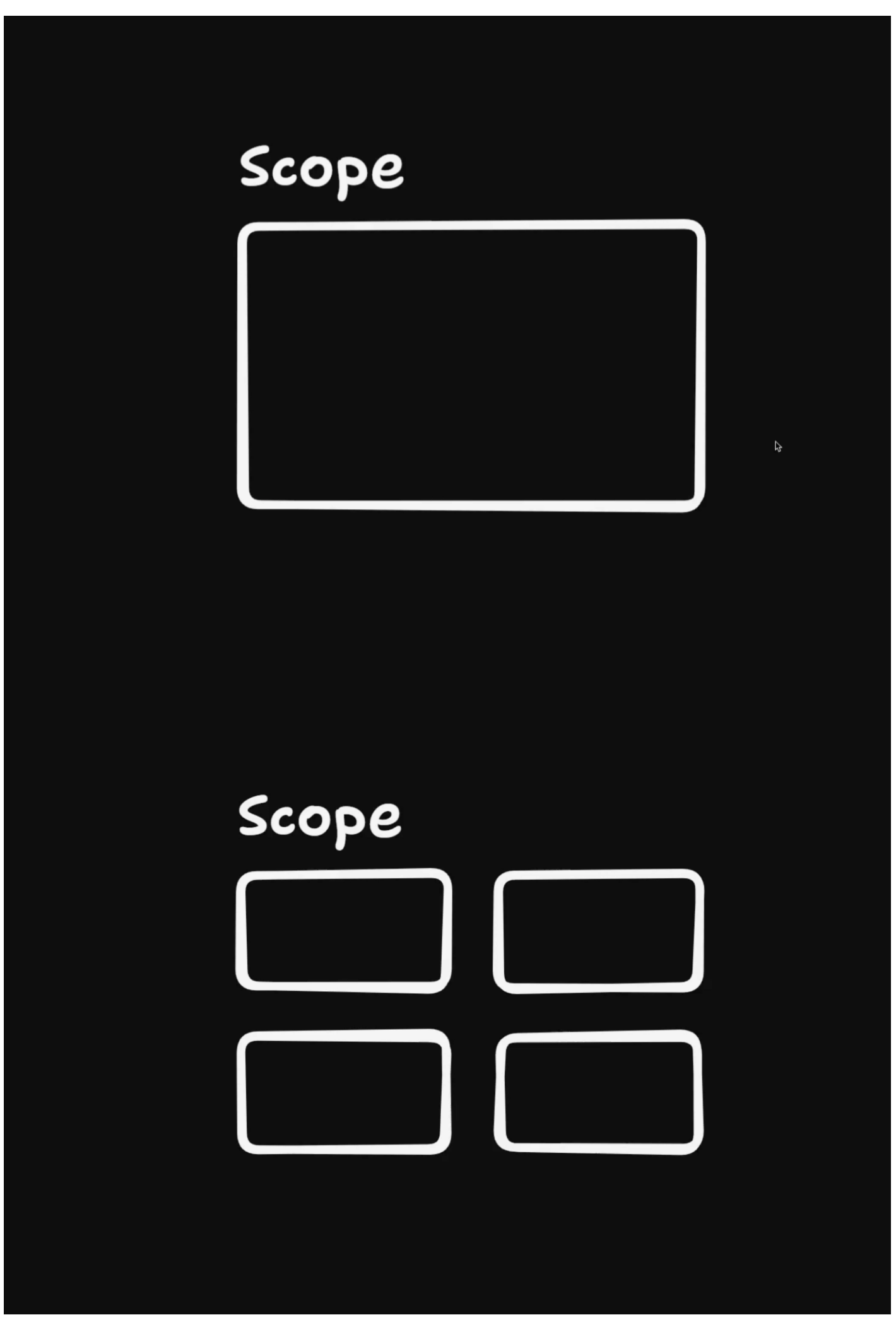


A refactor that touches every layer does not fit. It does not half-fit. It runs off the end of the window, and everything after the dashed line is produced by a model that has quietly stopped being good.



So you cover it with several sessions, each of which stays entirely in the smart zone. That is the whole argument for decomposition with an agent, and it is the same argument engineers have been making for decades under a different name.

Cutting is easy; cutting so each piece stands alone is not



Most people picture the scope of a change as one block and imagine the job is to draw lines through it. Cutting it is not the hard part. **Cutting it so that each piece can be executed without the others in the room** is the hard part, and that is what the rest of this section is about.

A unit of work is one a fresh session can execute

A **unit of work** is one that can be handed to a fresh agent session, with no other context, and executed from its own description alone. That test is the definition. Here is the same work twice:

- **Bad:** *"Add the returns export to ingestion, using the schema we agreed on. Handle the dates the way the other source does."* Every load-bearing word points somewhere the session cannot see — which schema, agreed when, which other source, handled how. It reads fine to you because you were in the room. A fresh session cannot ask; it guesses, and it guesses confidently.
- **Good:** *"In `ingest.py`, add a reader for `data/raw/returns_2026-08.csv`. Columns: `txn_id`, `store_id`, `txn_date` (`DD/MM/YYYY`), `amount` (negative, decimal comma), `quantity`. Return `RawRow`s with `source="returns"`, fields unparsed, exactly as the POS readers do. Done when `read_all` picks the file up and a test asserts the row count and the first row's fields."* It names the module, the input, the output shape, and how it knows it is finished. Nothing in it refers to this conversation.

A unit of work is not a unit test. The unit is the work; the document describing it is its **ticket**. When a fresh session asks for something the ticket does not contain, you fix the ticket.

Tracer bullets, and edges that are only preferences

Each unit should go end to end through the system rather than build one horizontal layer. A thin vertical slice that works beats a complete layer that cannot be run.

A **blocking edge** is a unit that genuinely cannot start until another finishes. *"B needs the table A creates"* is a true dependency. *"I'd rather do A first"* is a preference — it is not an edge, and calling it one is the most common mistake in this competency. The challenge question, applied out loud to every edge you draw: **what actually breaks if I go the other way?** If the answer is "nothing, it would just feel odd", it is a preference. If the answer is "the second unit has nothing to point at", it is an edge.

When the destination changes

Two tickets in, you review the work and realise the approach is wrong. The move is: **the tickets are disposable and the spec is editable.** Delete the tickets you have not implemented, edit the spec to describe the destination you now want, and regenerate the tickets from it. Usually you keep the code already shipped — it is real work, and reverting it is a separate decision.

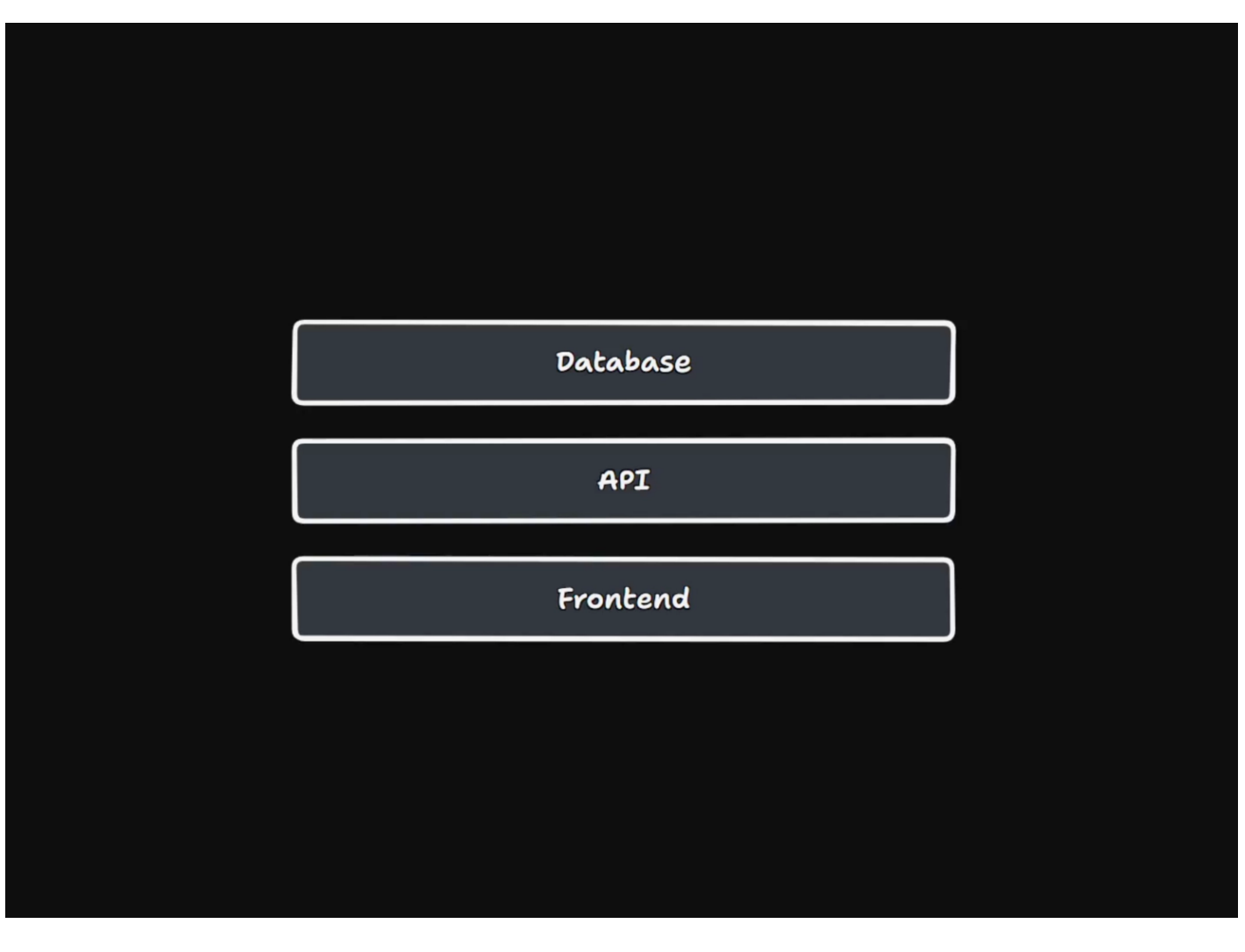
This is the reason the destination and the plan are two documents rather than one: they have **different lifetimes**. One gets edited; the other gets thrown away. A single document holding both makes it confusing to discard half of it, and that confusion is what keeps people executing a plan they have already stopped believing in.

The worked feature: getting the nightly pipeline off a laptop

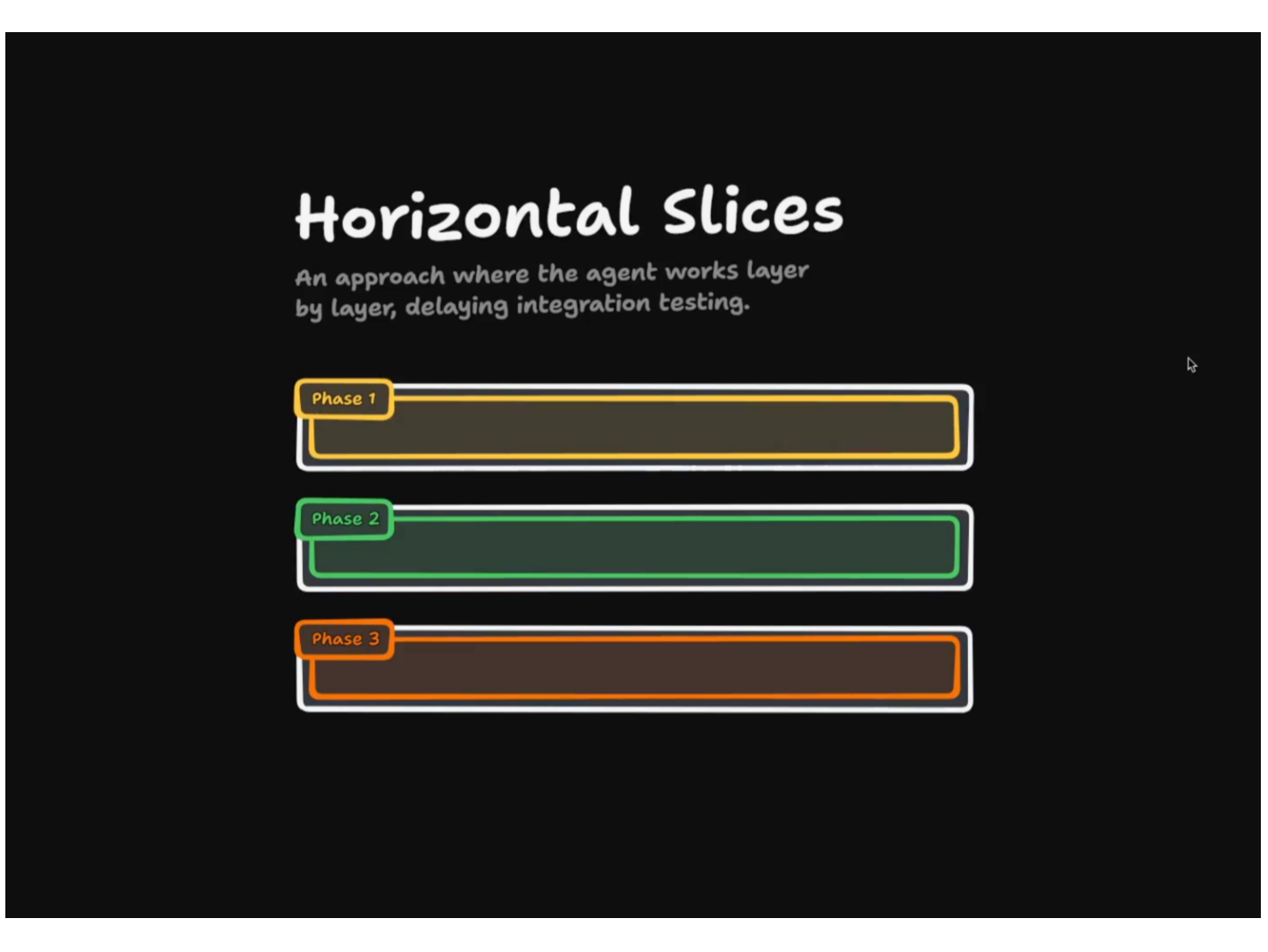
> **Feature A — Get the nightly pipeline off the analyst's laptop**
>
> *"The ingestion job needs to run itself. Put it in production."*
>
> - Today it is `python -m pipeline.run`, run by hand each evening by one analyst, against a checkout of `main`.
> - Credentials live in a `.env` file that has been forwarded on Slack more than once. There is a secret manager the other services already use.
> - GitHub Actions and a container registry are set up. Production runs scheduled work as Kubernetes CronJobs; several already run there.
> - When the job fails, nobody finds out until finance ask where the numbers are.

This is deliberately the same size and shape as a Lab 1 change: a fuzzy request, plus the handful of facts you need to decompose it without knowing the codebase. The room's first answers were layers — *containerise it → move the secrets → write the Kubernetes manifests → add alerting*. That cut is the one to argue with.

Layers are how the code is arranged, not how the work is cut

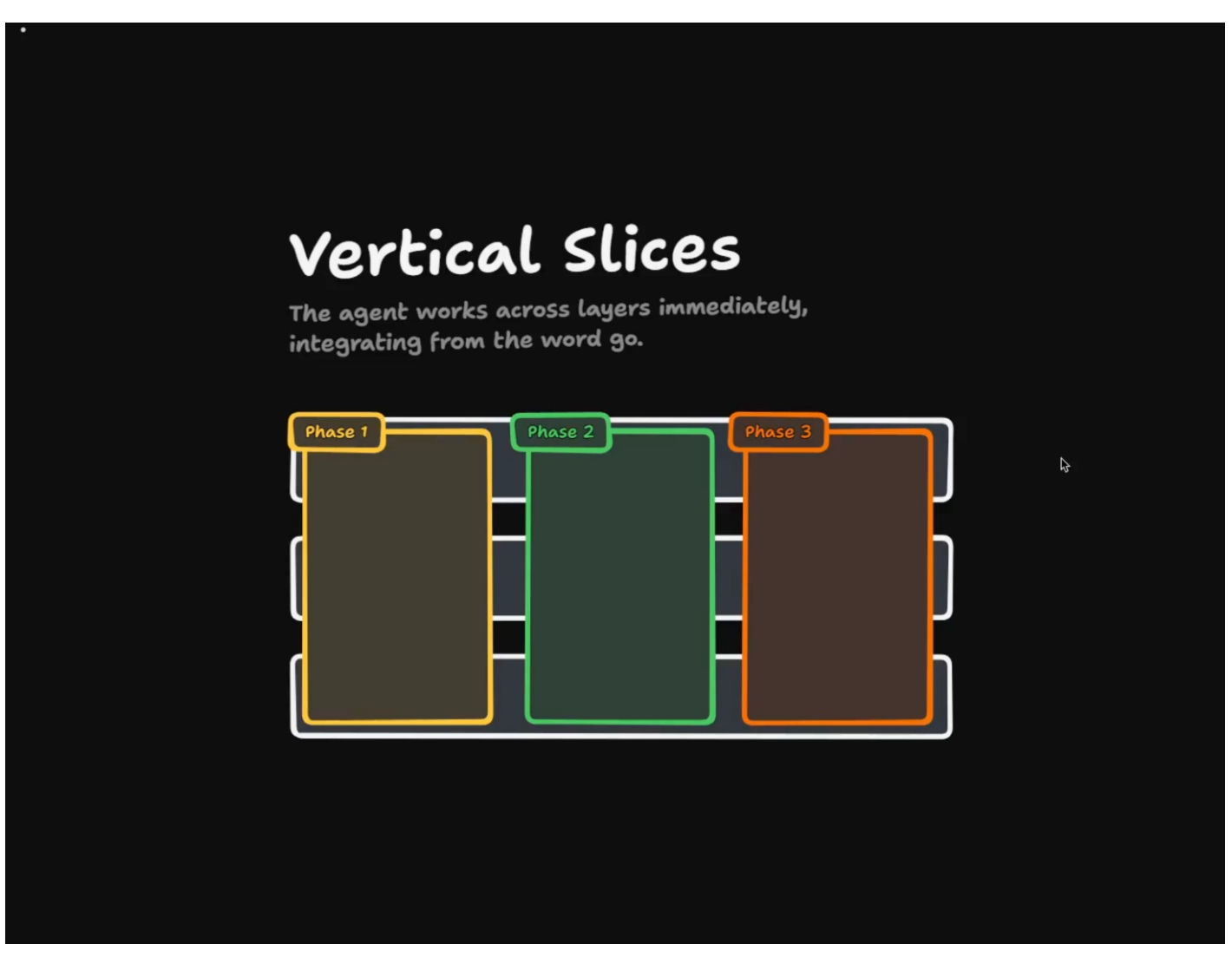


Three bars, stacked. For a web feature they are usually `Database` / `API` / `Frontend`; for a data or platform feature the names change but the shape does not. Build them top to bottom and you have a **horizontal slice**: the agent works layer by layer, and nothing is integrated — nothing has actually been *run* end to end — until the last bar is done.


Horizontal Slices
An approach where the agent works layer by layer, delaying integration testing.
Phase 1
Phase 2
Phase 3

The agent-specific cost is precise: a horizontal bar is finished when it compiles, not when it works. You cannot tell whether the agent got it right, so you cannot correct it, so the error travels — and by the time the last bar integrates, the session that wrote the first one is long gone and you are debugging a stranger's code with no memory of the decisions behind it. Feedback deferred is feedback you paid for and did not receive.

Vertical slices run


Vertical Slices
The agent works across layers immediately, integrating from the word go.
Phase 1
Phase 2
Phase 3

Cut the bars the other way: thin rectangles crossing all of them, each touching every layer and each one **running**. That is a unit, and it is the tracer bullet from earlier drawn as a picture.

> The bars are not wrong as a description of the system. They are wrong as a plan of work. Layers are how the code is arranged; units are how the work is cut. Confusing the two is the single most reliable way to end up with a feature that is 90% done for three weeks.

Feature A, cut vertically

A different cut of the same size and shape would be just as good. What matters is that each unit runs end to end and states how it knows it is finished.

1. **Run it once, in a container, from CI.** Dockerfile, image built and pushed on merge to `main`, entrypoint takes the run date. *Done when:* `docker run <image> --date 2026-08-21` reproduces what the analyst gets by

hand.

2. **Run it on schedule, in production.** CronJob using that image, reading credentials from the secret manager the other services use. *Done when:* it has fired on schedule once, unattended, with no `.env` in the container.
3. **Tell someone when it fails.** Non-zero exit routes to the channel the analyst and finance already watch, with the run date and a link to the logs. *Done when:* a deliberately failing run produces the alert.
4. **Close the old door.** Rotate the credentials that went over Slack, delete the forwarded `.env`, write the runbook entry for a manual re-run. *Done when:* the old secret is revoked and the documented re-run works.

Notice what unit 1 does: it is the smallest thing that touches every bar, and it is where the decision that makes the rest safe gets taken — the container entrypoint. Vertical slices are not only about feedback; they force the load-bearing decision to the front.

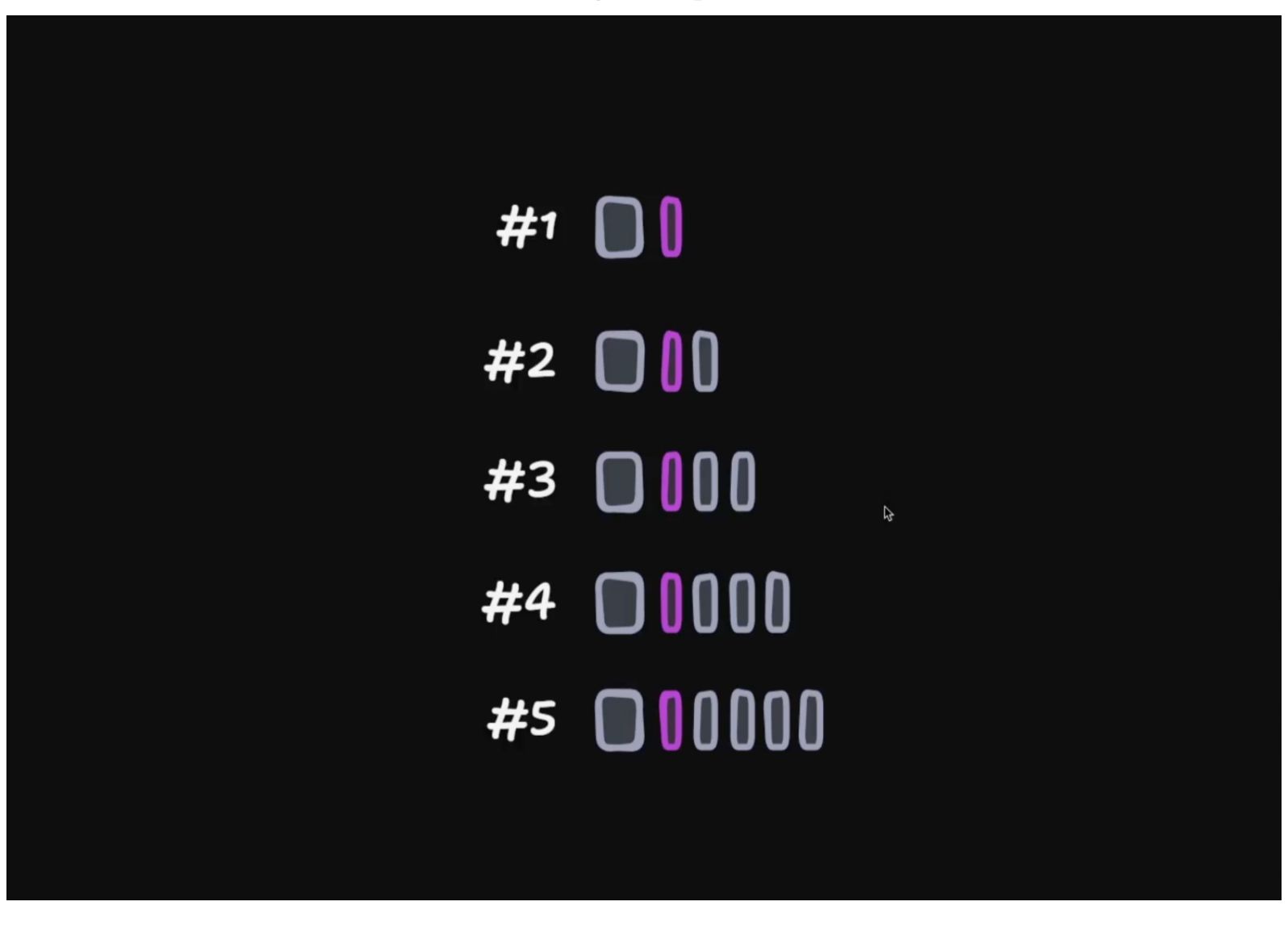
The edges of Feature A, one at a time

- **True:** 1 → 2. The CronJob has to name an image that exists. There is nothing to schedule until something is published.
- **Preference:** 2 → 3. "Alerting comes after it's running" sounds obvious and is not an edge — alerting is built against a job that fails on purpose, which you can do on day one. Doing it last is the reason nobody found out about the failures in the first place.
- **True, and the interesting one:** 2 → 4. Rotating the credential before the production job reads from the secret manager breaks the analyst's manual run, which is still the only thing producing the numbers. The edge is not technical; it is that you cannot remove the fallback until the replacement works.

> A false edge costs you the thing you came for. True edges are the only reason to work in sequence; everything else can be a separate session, a separate branch, a separate agent. Every edge you draw that is really a preference is parallelism you have thrown away — and, worse, a unit you will now write with the previous one still in your head, which is exactly how the context-dependent ticket in the "bad" example gets written.

The grid asks for three or more units and at least one true blocking edge, and the oral asks you what breaks if it is violated. (Grid: C1 Solid.)

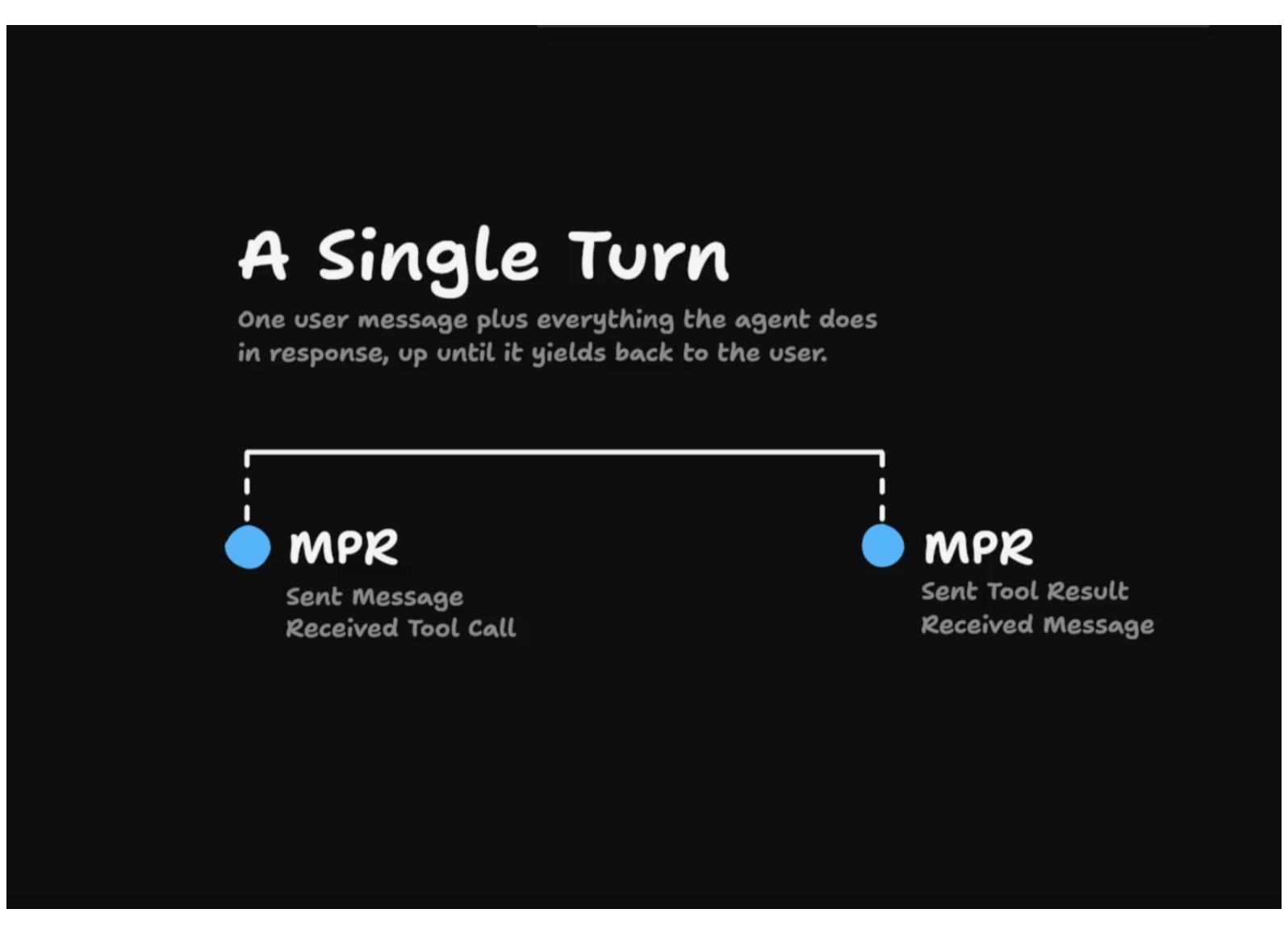
Context is re-sent on every request

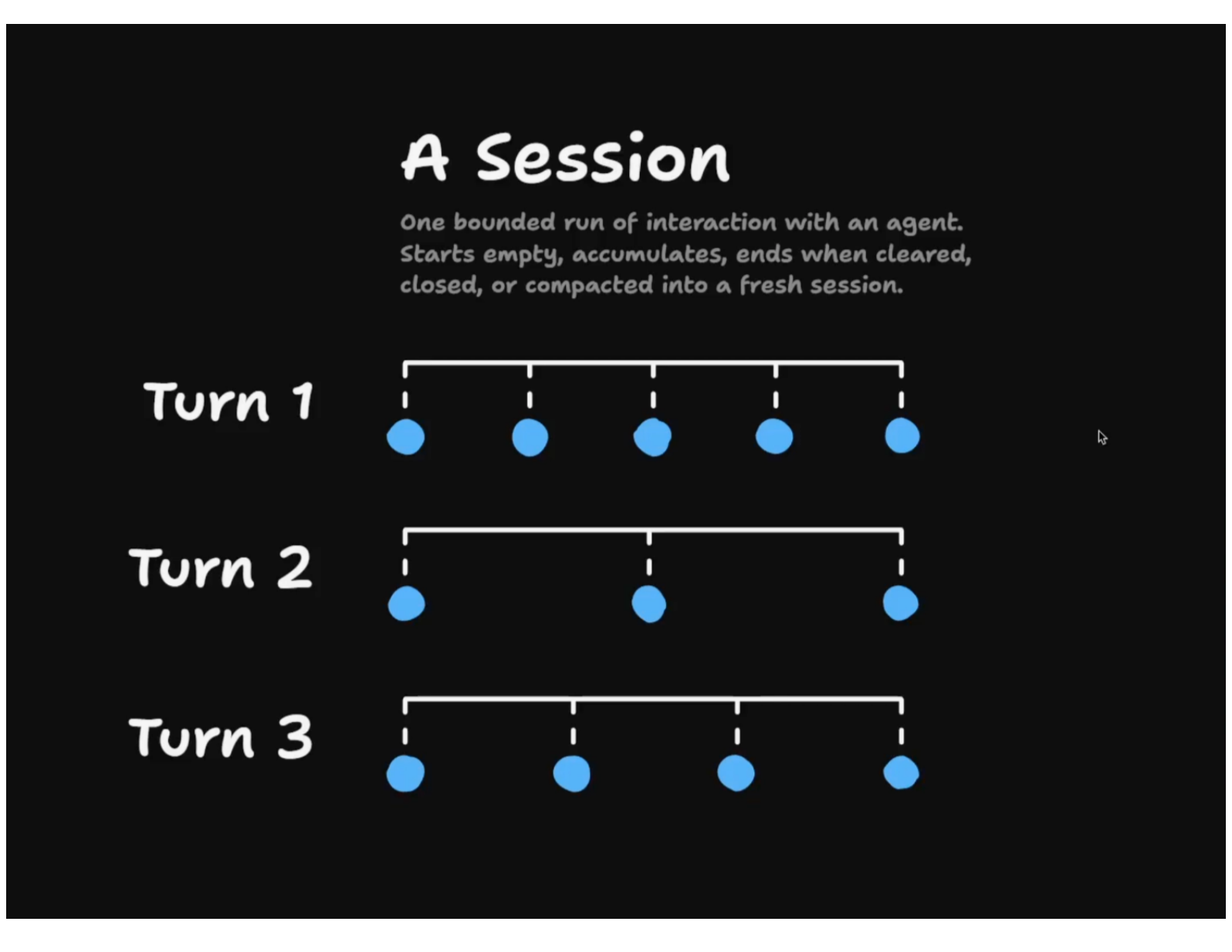


A conversation is not something the model remembers. Every request re-sends the entire history from scratch, so request five carries everything from requests one through four. Nothing is retained on the other end; it is re-uploaded. That is what "context" means, and it is why a long session gets slower and more expensive as well as worse.

Request, turn, session

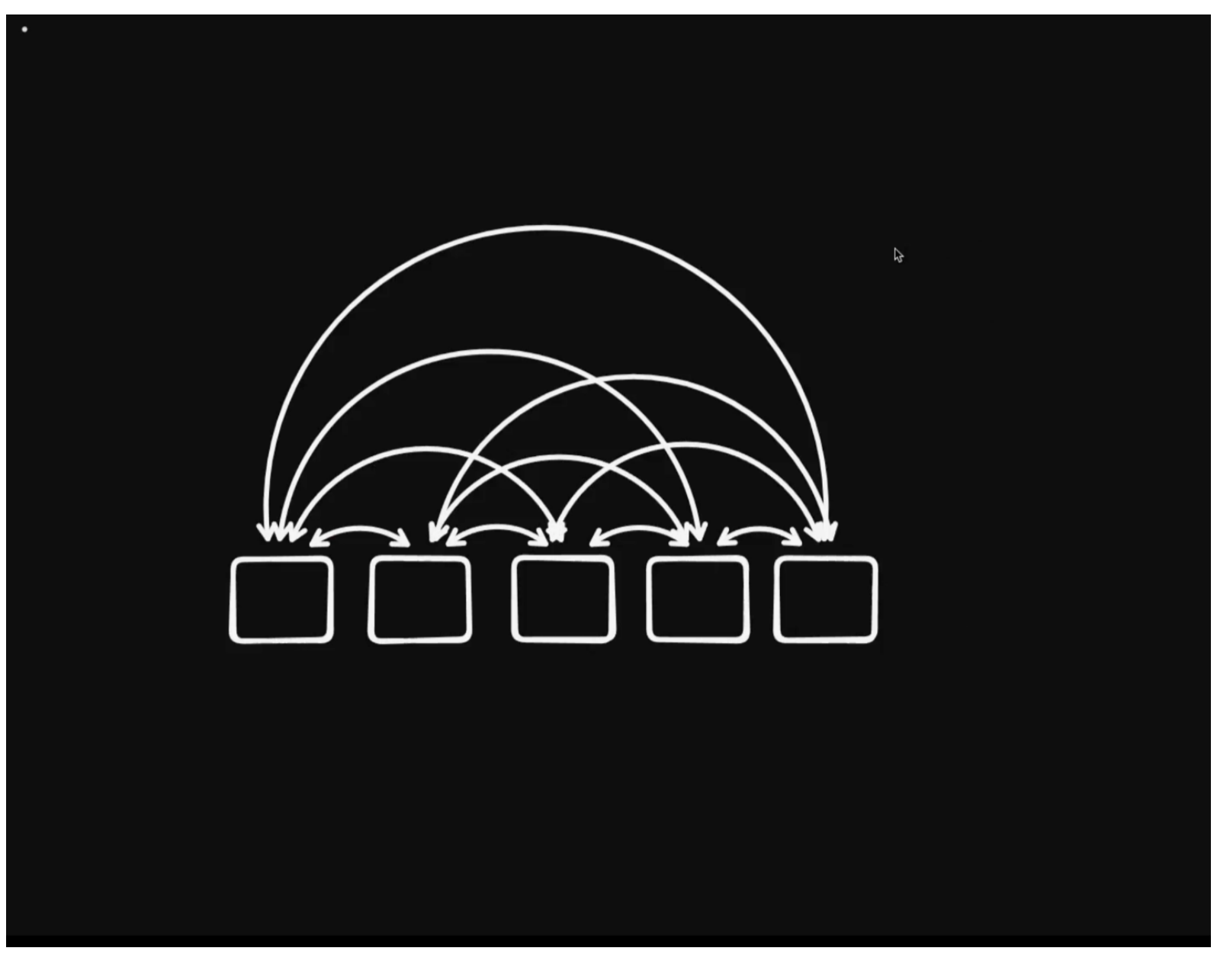
Model Provider Request


A Single Turn
One user message plus everything the agent does in response, up until it yields back to the user.
MPR
Sent Message
Received Tool Call
MPR
Sent Tool Result
Received Message

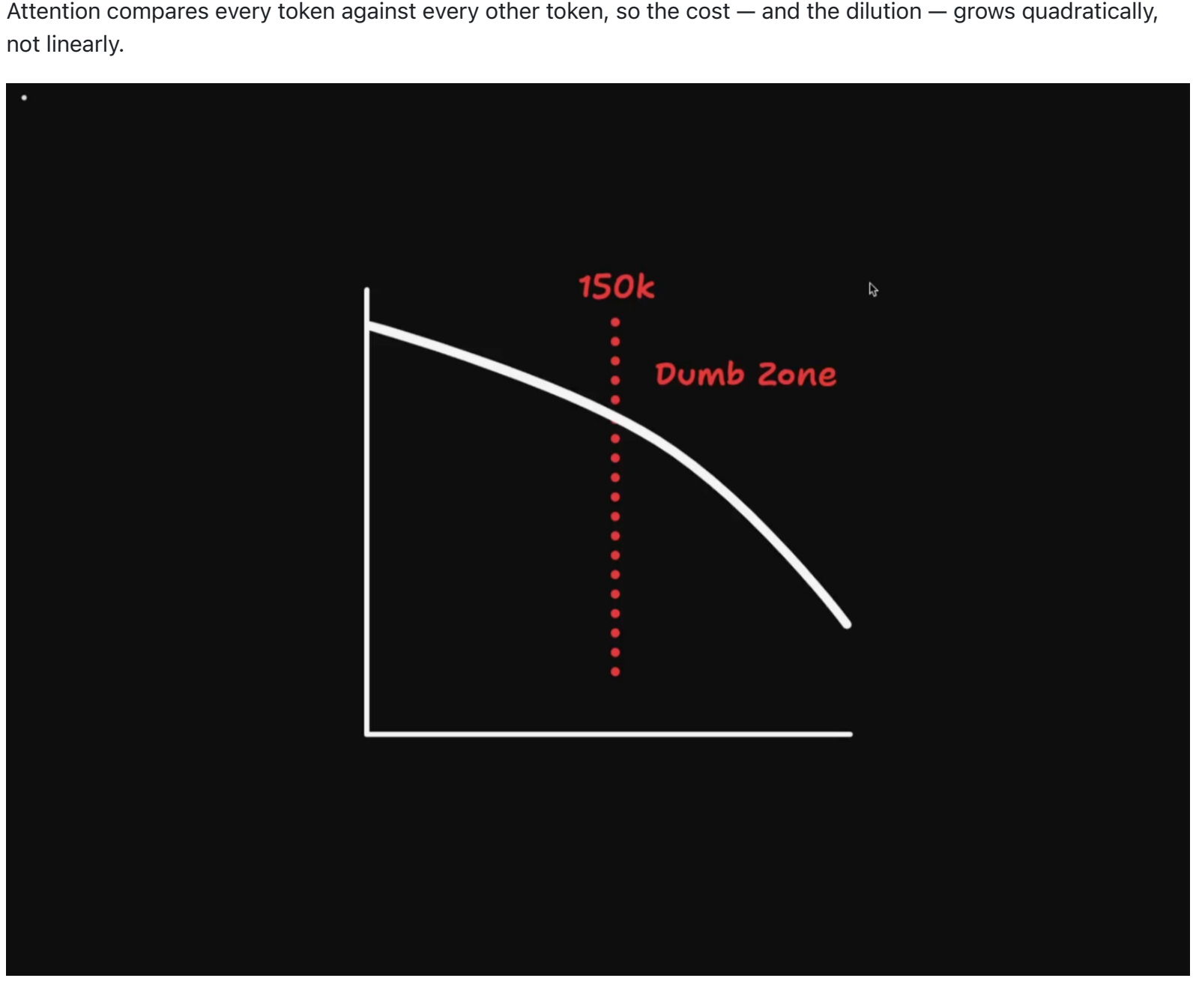

A Session
One bounded run of interaction with an agent. Starts empty, accumulates, ends when cleared, closed, or compacted into a fresh session.
Turn 1
Turn 2
Turn 3

One **request** is one round trip to the model. One **turn** is your message plus everything the agent does before it hands control back — often a dozen requests. One **session** starts empty, accumulates every turn, and ends when it is cleared, closed or compacted. The unit that fills up is the session; the unit you are billed for is the request; the unit you experience is the turn. Confusing them is why "I only sent three messages" and "you spent 200k tokens" are both true.

Why it degrades before it is full

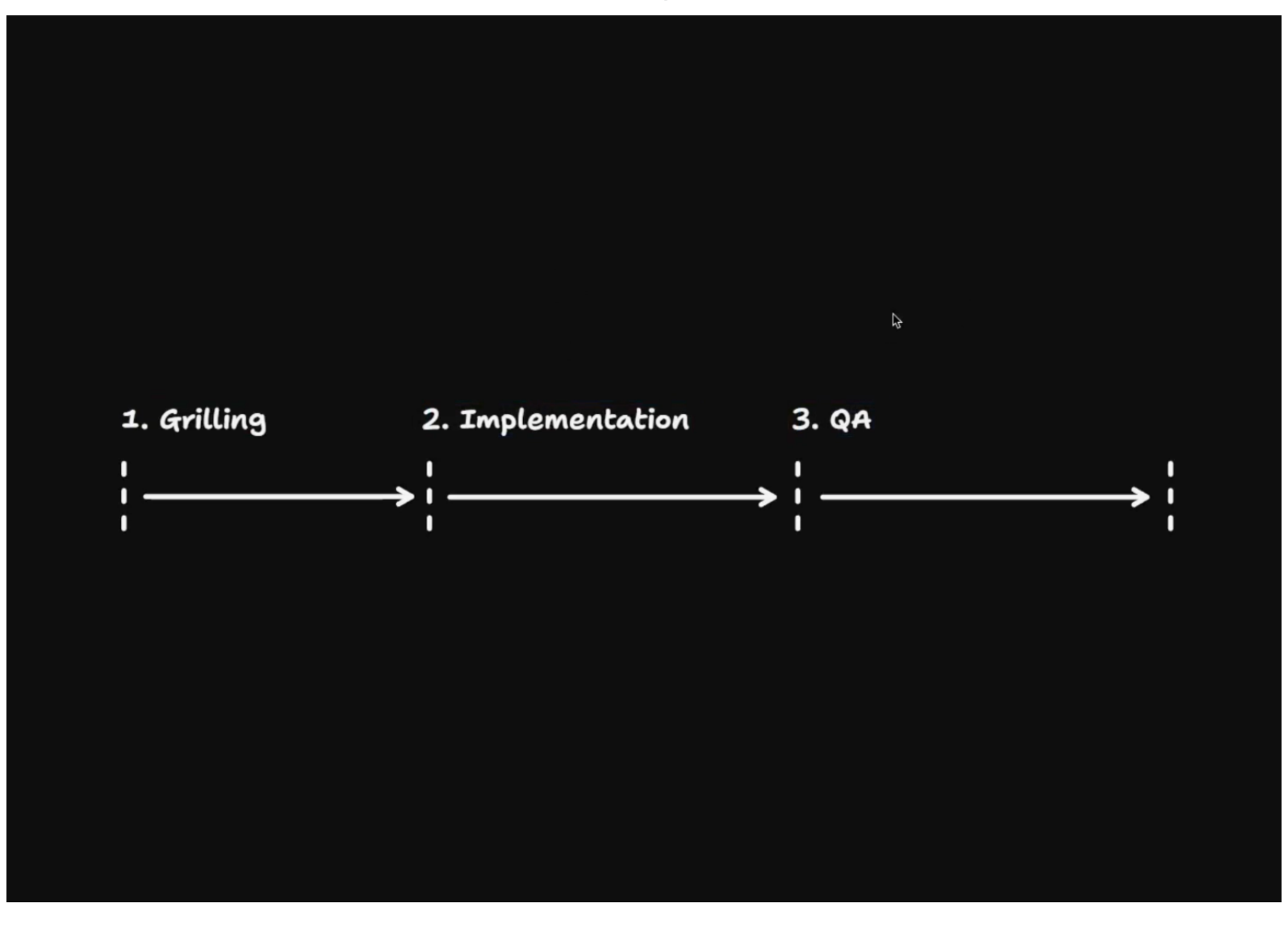


Attention compares every token against every other token, so the cost — and the dilution — grows quadratically, not linearly.



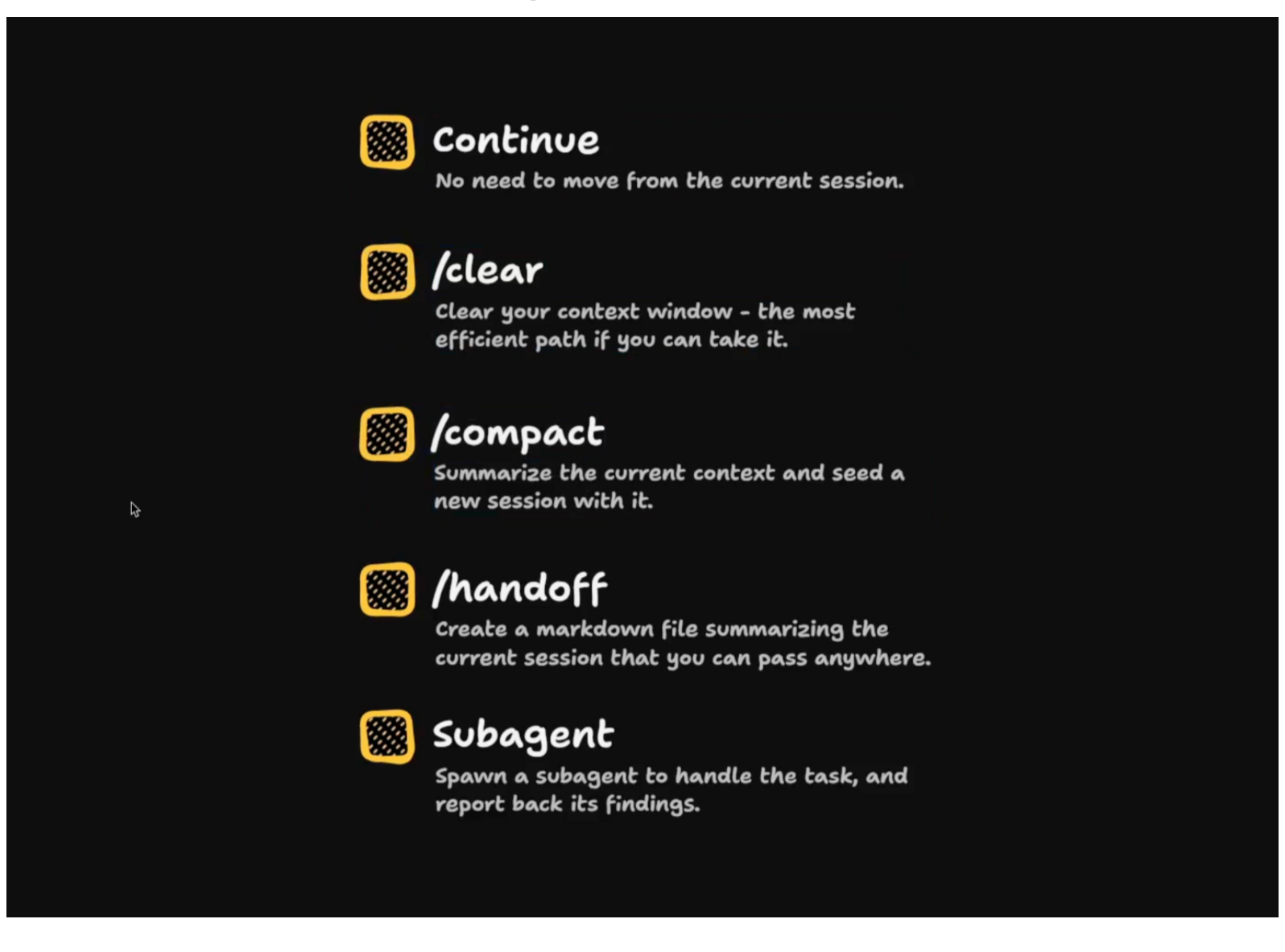
A model reasons sharply within a window — roughly 150k tokens on current frontier models — and degrades before it runs out. Degradation is silent: there is no error, the answers just get worse. This is why "it was going great and then it got stupid" is the single most common report.

Phase boundaries are where you decide


1. Grilling
2. Implementation
3. QA

A phase is a chunk of work: the framing, the build, the review. The decision about context belongs at the boundary between two of them, and nowhere else. Mid-phase, your only real options are to continue or to split the remainder off.

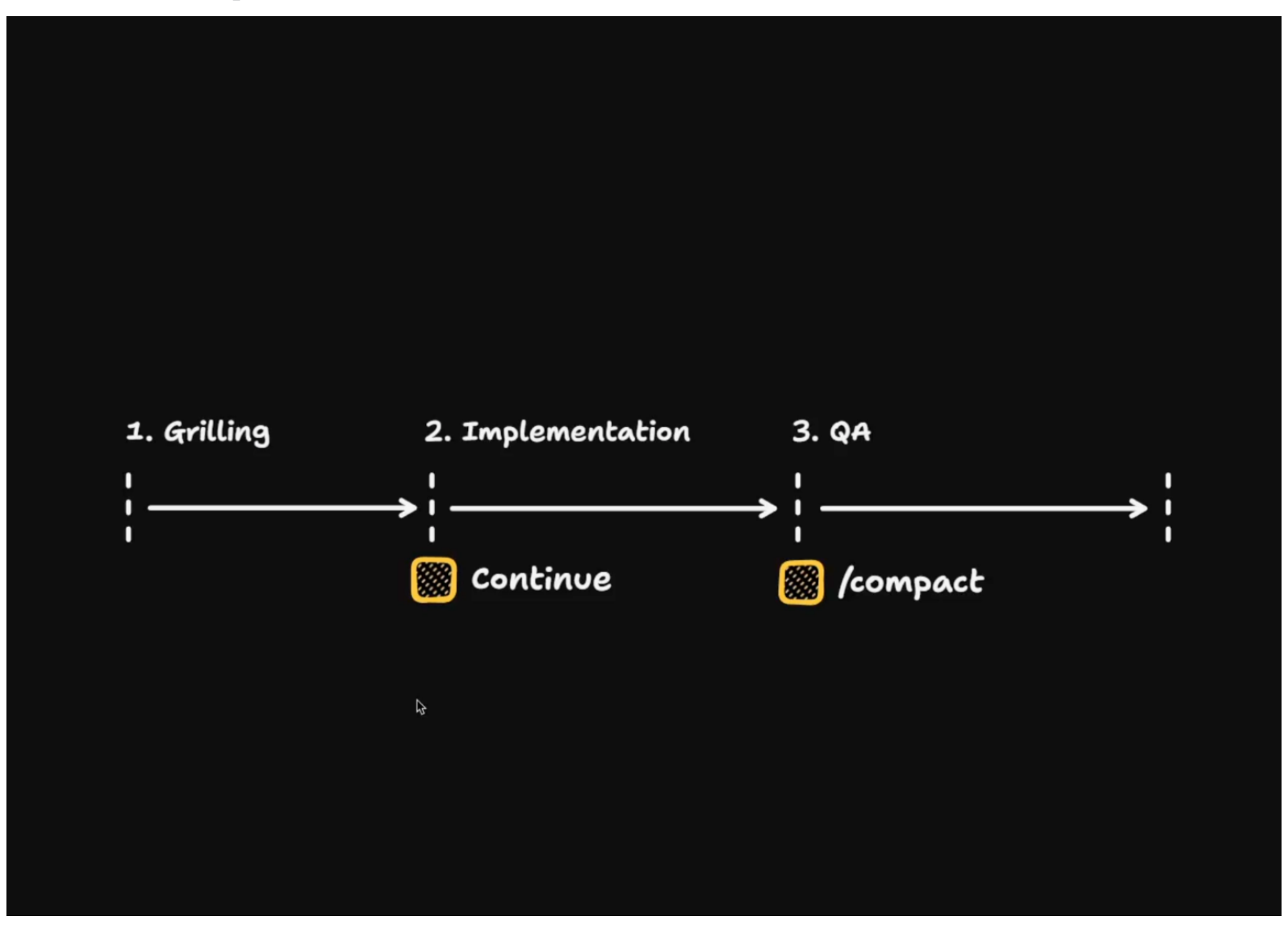
The five context moves, ranked


Continue
No need to move from the current session.
/clear
Clear your context window - the most efficient path if you can take it.
/compact
Summarize the current context and seed a new session with it.
/handoff
Create a markdown file summarizing the current session that you can pass anywhere.
Subagent
Spawn a subagent to handle the task, and report back its findings.

They are ranked, cheapest and most faithful first, and the order is the lesson. Compaction is at the bottom because it is the only one of the five that destroys evidence.

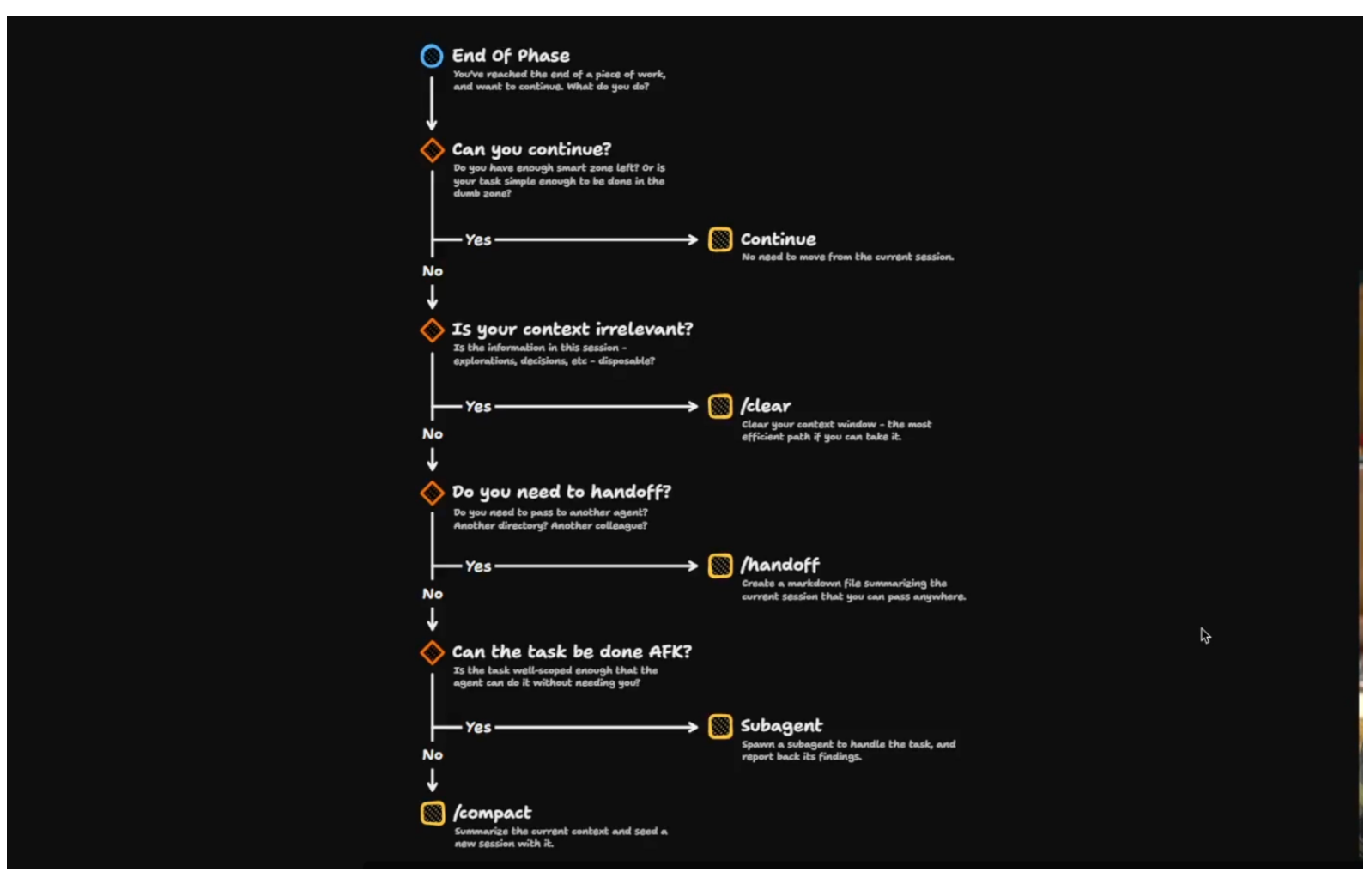
- **Continue** — costs nothing, loses nothing. Rule it out first, deliberately: if there is smart zone left, or the next phase is simple enough not to need much, you are done deciding.
- **Clear** — nothing here matters to what is next. The most efficient path available, and the one nobody takes, because throwing away a conversation you just had feels like waste. It is not: if the output you needed is already written down, the conversation that produced it is disposable.
- **Hand off** — write a portable document, for a new machine, a new directory, a colleague, or tomorrow. It costs a few minutes and buys a primary-source record, and it is the only one of the five that crosses a boundary a compaction cannot: compaction works inside one agent in one directory, so **moving work from Claude Code to Codex, or to a machine that has never seen this session, is a handoff or it is nothing.** Put the document somewhere disposable — a handoff is a bridge between two sessions, not project documentation. Commit it and you have manufactured exactly the stale secondary source you were told to archive.
- **Subagent** — send a scoped task to its own window, get a report back. Available whenever the task is well-scoped enough to run without you in the room.
- **Compact** — compress and carry on. **The fallback**, for when none of the four above apply: you cannot continue, the context is not disposable, it is going nowhere else, and it needs you.

A worked pass down the list


1. Grilling
2. Implementation
3. QA
Continue
/compact

At the framing → build boundary there is smart zone left, so **continue** and stop thinking. At the build → QA boundary you cannot continue; the context is not disposable, because QA needs to know what was decided and why; it is not going to another machine or another person; and QA is a conversation you want to be in. Four rule-outs, so **compact**. Note what makes that answer defensible — not that compacting is what one does, but that the other four were tried and failed.

The same list as a procedure


End Of Phase
You've reached the end of a piece of work, and want to continue. What do you do?
Can you continue?
Do you have enough smart zone left? Or is your task simple enough to be done in the dumb zone?
Yes
Continue
No need to move from the current session.
No
Is your context irrelevant?
Is the information in this session - explorations, decisions, etc - disposable?
Yes
/clear
Clear your context window - the most efficient path if you can take it.
No
Do you need to handoff?
Do you need to pass to another agent? Another directory? Another colleague?
Yes
/handoff
Create a markdown file summarizing the current session that you can pass anywhere.
No
Can the task be done AFK?
Is the task well-scoped enough that the agent can do it without needing you?
Yes
Subagent
Spawn a subagent to handle the task, and report back its findings.
No
/compact
Summarize the current context and seed a new session with it.

> Compaction is where you end up when you have ruled out everything cheaper — not where you start.

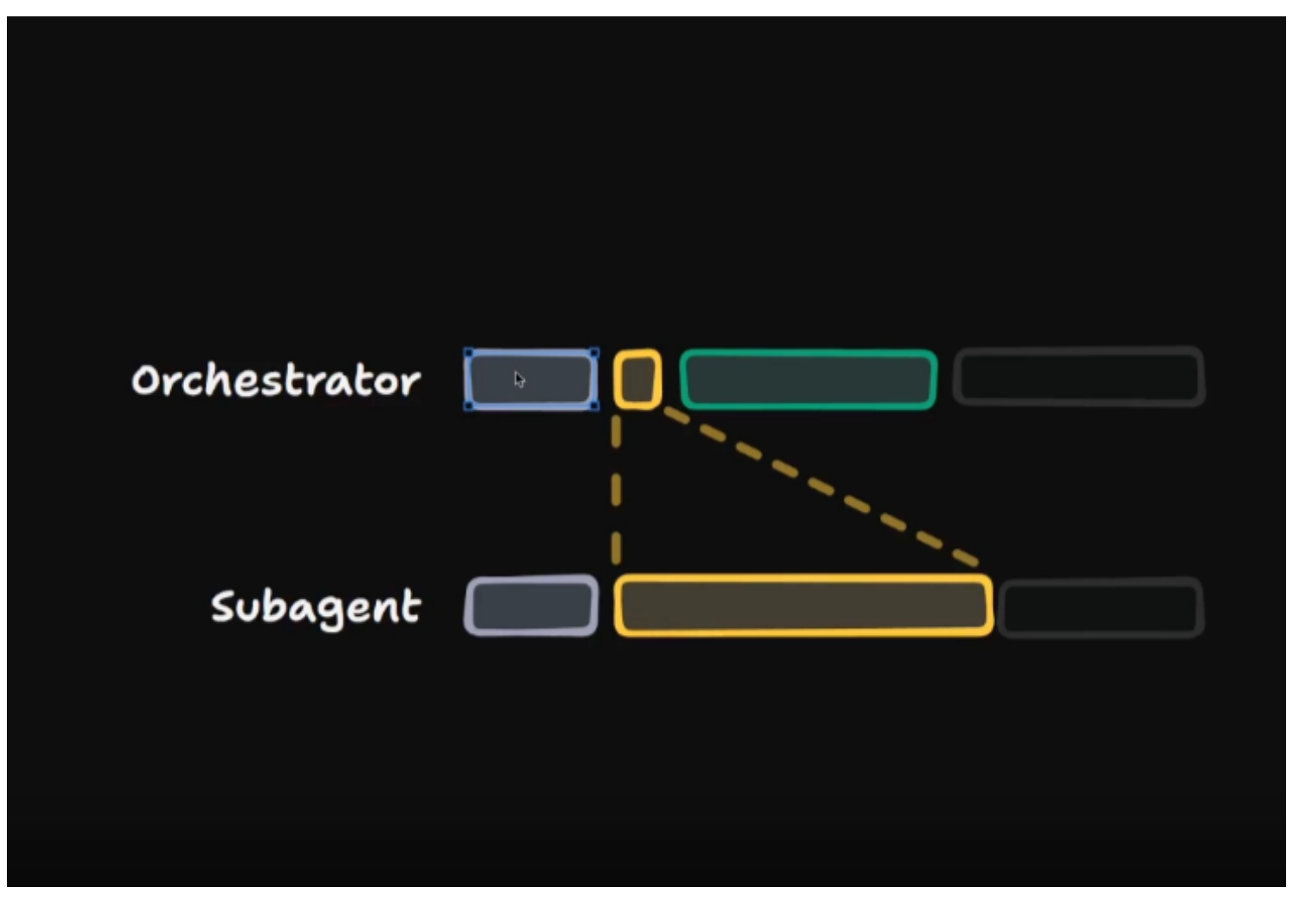
Why not just compact

Compaction is the move everyone already reaches for, and the harness offers it automatically, so the argument against starting there is worth stating plainly.

- It is the **only one of the five that loses information you cannot get back**. Continue keeps everything. Clear keeps everything that was written down. Handoff and subagent both produce an artifact. Compaction produces a summary and deletes its own source.
- It is also the only one the harness will do to you **without asking**, when the window fills. Autocompaction is the same move taken by nobody, at the worst possible moment, usually mid-phase. If you are regularly being autocompacted, you are not managing context; the context is managing you.
- And it is the most **expensive** of the five in the thing you actually care about: quality after the boundary. Clearing starts the next phase at full sharpness from a written source. Compacting starts it from a lossy retelling.

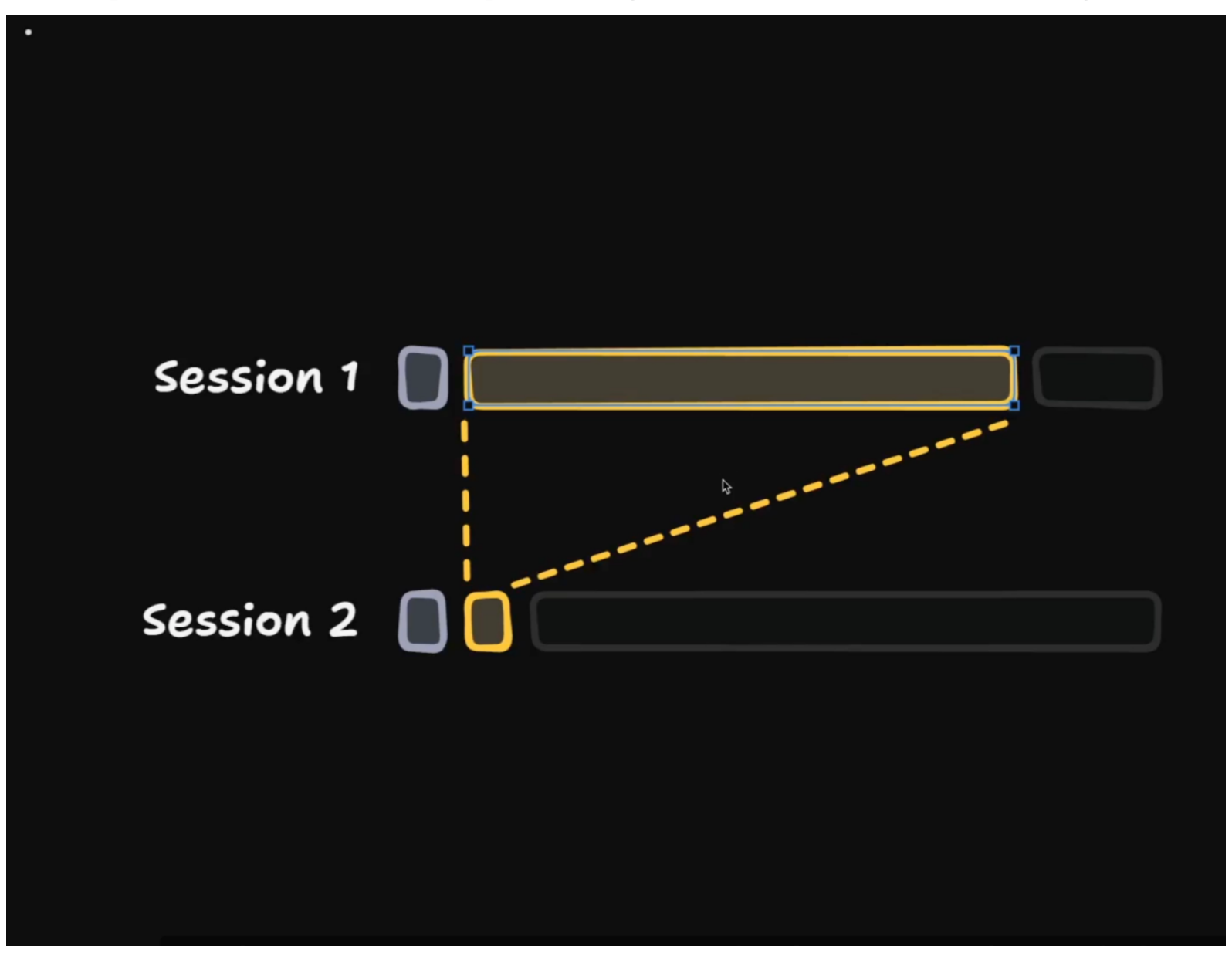
The grid asks for a recorded context decision, and *"I compacted"* on its own is a Basic answer. What moves it to Solid is naming what you ruled out — the same shape as C3's "what did it rule out?". The two competencies are asking the same question.

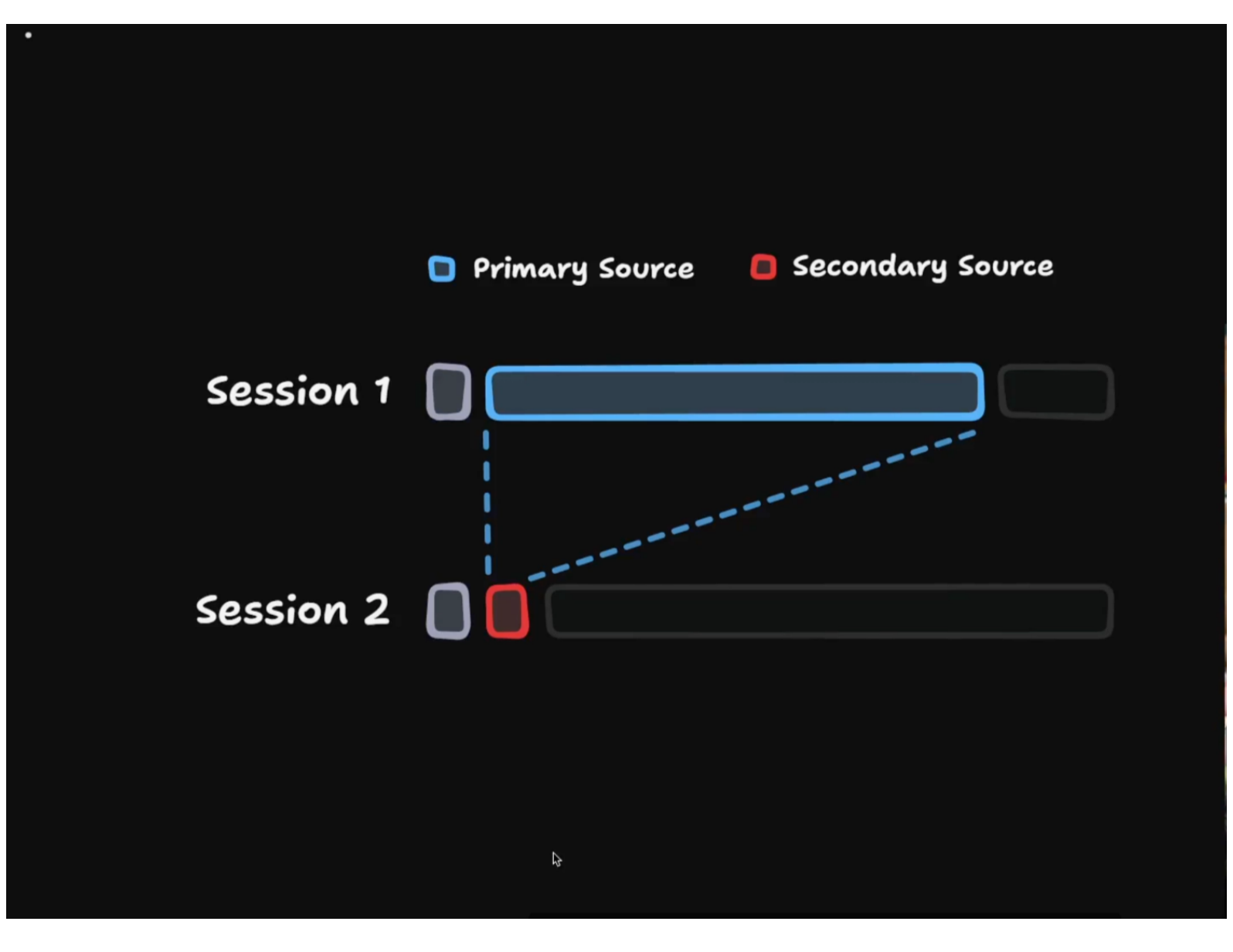
What a subagent actually does


Orchestrator
Subagent

A subagent spends its own window on the work and hands back a summary, so the parent's context barely moves. This is the move for anything long, well-scoped, and not interesting in detail — a search, a survey, a review pass.

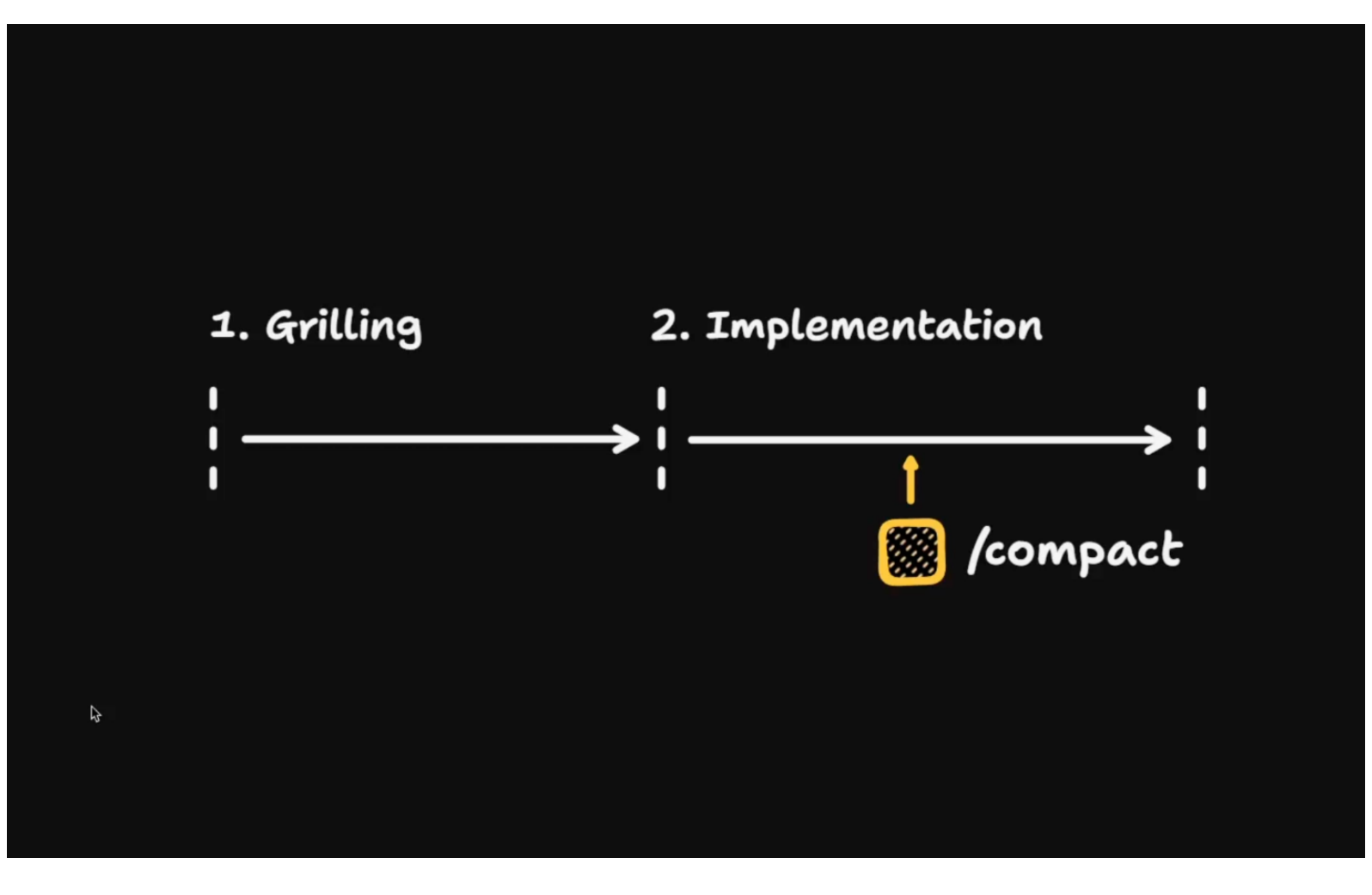
Compaction demotes primary source to secondary


Session 1
Session 2


Primary Source
Secondary Source
Session 1
Session 2

This is the whole reason compaction sits last on the list. What survives compaction is not the conversation — it is a retelling of the conversation, written by the same model that is about to rely on it. Every compaction demotes primary source to secondary, and a second compaction is a retelling of a retelling. Anything load-bearing should have been written to a file before the boundary, which is exactly what the specification and the tickets are for — and note what that does to the ranking: **the better your written artifacts, the further up the list you can afford to move.** A team with real specs and tickets clears where a team without them has to compact.

Compacting mid-phase is the one move to name as a mistake


1. Grilling
2. Implementation
/compact

You lose the information **and** the rails it was running on: the agent comes back with a summary of what it was doing and no grip on how it was doing it. If you are drowning mid-phase, the phase was too big — that is a decomposition problem, and you fix it in the ticket, not in the context.

Model and effort, in three sentences

The fiche's competency statement says *"an efficient combination of model and effort level"*, and here is the whole of it: **pick a top-tier model, set effort to medium, and stop touching both.** Max effort exists so a provider can win a benchmark by two points, not so you can have a better Tuesday. Raising effort is not free in the currency this lecture has spent its time on — reasoning tokens are tokens, so a high-effort session reaches the dumb zone sooner and therefore fits **less** work. Effort is a decomposition variable.

The punchline is the entire course in one sentence: when the output is not good enough, the leverage is almost never in the model. It is in the harness and the environment — the spec, the ticket, the instruction document, the context you did or did not spend. Model-swapping is the thing everybody reaches for first because it is the only knob that requires no thought.

Model, effort, and what a session costs has the rest: how to A/B two setups honestly, why cost per task beats price per token, and what prefix caching does to the bill. It is read-once material with a two-year shelf life, none of it is graded, and it is a document precisely so that it does not eat lecture time.

What happens next

Lab 1 is the first three phases of the method — grill, spec, tickets — on a repository of your own. Nothing in the **Resources** section is required before anything; the documents are there to be read after the session that sent you to them.

The pictures in this lecture are from the AI Hero crash course.

Modifié le: samedi 29 août 2026, 21:35